

GMP 及 IRIS 主控板的使用说明

李戟琿

目录

1 GMP 提供的软件资源	1
1.1 控制器模板资源.....	1
1.2 代码规范概述	3
1.3 平台提供的几项关键例程	7
2 GMP 提供的仿真资源	7
3 GMP 平台安装、部署说明	8
3.1 GMP 软件平台下载和安装.....	8
3.2 基于 Simulink 的仿真环境搭建.....	12
3.3 基于 CCS 的 C28x 系列 DSP 环境.....	14
3.4 基于 STM32 的开发环境安装.....	14
4 基于 Git 管理项目代码.....	15
4.1 Git 功能概述.....	15
4.2 以 Suite 为单位管理自己的工程项目	15
4.3 基于 git 仓库的小组协作.....	18
4.4 几项关键的 git 操作	19
5 基于 DSP280039C 的 IRIS Node 主控板说明	20
5.1 DSP 的常用外设操作	20
5.2 GMP 的数字控制器基本结构	24
5.3 FPGA 模块主要功能说明	26
6 训练任务	27

1 GMP 提供的软件资源

GMP 库提供了丰富的控制器模块（component）和成套的控制器模板（suite）资源。用户可以基于控制器模块完成复杂的控制器模块化设计，也可以基于成套的控制器模板快速地建立工程、落地项目，还可以根据科研需要基于 GMP 的跨平台设计添加自己的高性能控制器模块。

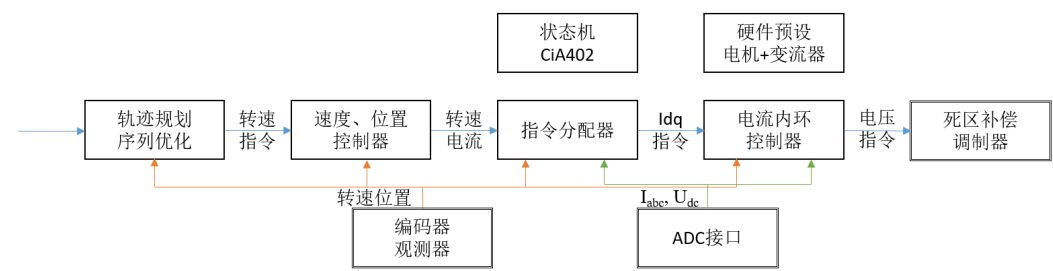
1.1 控制器模板资源

在 gmp_pro\ctl\suite 目录下可以找到大量的控制器模板，可以选择其中任意一个开始控制器设计、调试、部署之旅。

User_main	用户交互逻辑	平台无关、实时性不重要、业务逻辑、速度慢、体量大 保存在工程目录下的common文件夹中，各平台共用。
Ctrl_main	控制器逻辑/保护逻辑	平台无关、严格实时任务、控制逻辑、速度快 保存在工程目录下的common文件夹中，各平台共用。
xplt.peripheral.c	外设初始化与操作	平台相关、实时性不重要、初始化、差别大 保存在工程目录下的f280049文件夹中，平台专用，用来初始化外设、负责平台/外设相关的任务和接口。
csp	芯片特性与功能	平台相关、实时性不重要、初始化、提供外设服务 保存在gmp_pro/csp/c28x_syscfg文件夹中，与芯片、编译环境、编程环境相关。
GMP Core	GMP提供的基础服务	平台无关、跨平台框架、进程服务、内存管理、控制模块 保存在gmp_pro/csp/c28x_syscfg文件夹中，与芯片、编译环境、编程环境相关。
CTL Module	CTL提供的控制功能	平台无关、控制模块、控制器框架、数学支持、工程例子 保存在gmp_pro/ctl文件夹中，其中math_block提供数学计算的支持，framework提供控制框架，component提供控制模块，suite提供例子。

首次运行时，需要运行工程根目录下的 gmp_src_mgr 文件夹中的 gmp_fac_generate_src.bat，这个文件将会把必要的支持库文件复制到

其中，mcs_pmsm_nt 工程适用于本手册。



GMP 的 CTL 库中提供了丰富的控制器模块功能，可以在 ctl/component 中找到，各个控制模块都有数据结构和功能函数组成。

1.1.1 基础模块 (Basic)

basic 子模块提供了一系列在控制系统中不可或缺的基础功能。

分频器 (divider.h): 实现了一个基于计数器的分频器, 用于从高频事件中生成低频触发信号, 适合在快速中断中执行慢速任务。

饱和/限幅模块 (saturation.h): 提供标准的饱和(限幅)功能, 可将信号强制限制在指定的上下限之间, 并支持为正负信号设置不同限制范围的双极性饱和模式。

斜率限制器 (slope_limiter.h): 也称为变化率限制器, 用于约束信号的变化速率, 防止输出发生突变, 常用于实现平滑的设定值过渡。

1.1.2 连续控制器 (Continuous Controller)

本模块包含基于连续时间理论设计的经典控制器。

连续形式 PID 控制器 (continuous_pid.h): 提供了基于连续 PID 公式离散化后的标准 PID 和抗积分饱和 (Anti-Windup) PID 控制器, 支持串联和并联两种形式。

跟踪 PID 控制器 (track_pid.h): 这是一个复合控制器, 它将分频器、斜率限制器和连续 PID 控制器相结合, 专为需要平滑设定值跟踪的场景设计。

二阶广义积分器 (sogi.h): 实现了一个 SOGI (Second-Order Generalized Integrator), 它是一种频率自适应滤波器, 能够从输入信号中生成同相和 90° 相移 (正交) 的正弦信号, 是锁相环 (PLL) 等应用的核心。

S 域传递函数 (s_function.h): 提供了一个通用的二阶 S 域传递函数模块。用户可以通过直观地指定 S 平面上的极点和零点频率来快速设计和实现一个控制器或滤波器。

1.1.3 离散控制器 (Discrete Controller)

本模块提供了直接在离散时间域 (Z 域) 中设计的丰富多样的控制器、滤波器和信号发生器。

离散 PID 与跟踪 PID (discrete_pid.h, track_discrete_pid.h): 提供了标准的增量式离散 PID 控制器, 以及集成了斜率生成和分频功能的复合跟踪 PID 控制器。

谐振与比例谐振控制器 (proportional_resonant.h): 实现了多种谐振控制器 (R, PR, QR, QPR), 它们能在特定谐振频率提供极高增益, 是实现交流信号无静差跟踪的关键, 尤其适用于并网逆变器电力电子应用。

离散滤波器与补偿器 (discrete_filter.h, pole_zero.h, lead_lag.h): 提供了丰富的线性滤波器库, 包括一阶/二阶 IIR 滤波器、可配置的通用极零点补偿器 (1P1Z, 2P2Z, 3P3Z) 以及经典的超前/滞后补偿器, 用于系统校正和噪声抑制。

通用 Z 域传递函数 (z_function.h): 这是一个通用的 IIR 滤波器模块, 用户可以通过直接提供 Z 域传递函数的分子和分母系数, 来实现任意的线性时不变控制器或滤波器。

离散 SOGI (discrete_sogi.h): 离散形式的二阶广义积分器, 功能与连续版本类似, 是数字锁相环 (PLL) 和高级信号分析的基础。

信号发生器 (signal_generator.h): 包含用于仿真、测试和系统激励的常用信号发生器, 如基于相量旋转法的高效正/余弦波发生器和斜坡信号 (锯齿波) 发生器。

1.1.4 高级控制器 (Advanced Controller)

本模块提供了更复杂的控制算法和工具，用于实现自适应控制等高级功能。

查表与插值 (surf_search.h): 提供了实现复杂非线性函数所需的基础工具——查找表 (LUT)。它支持一维和二维 (均匀/非均匀网格) 的查表，并内置了高效的搜索 (二分法) 和插值 (线性/双线性) 算法，是实现高性能非线性控制的基础。

模糊 PID 控制器 (fuzzy_pid.h): 实现了一个模糊自整定 PID 控制器。它利用误差(E) 和误差变化率(EC)作为输入，通过查询预设的二维查找表 (即模糊规则) 来实时调整 PID 的 K_p , K_i , K_d 参数，从而让控制器能够动态适应系统特性的变化。

1.2 代码规范概述

为了保持代码的一致性和可读性，本库中的所有函数均遵循统一的命名规则。这有助于开发者快速理解函数的功能和使用场景。

基本结构:

<code>ctl_<action>_<target_object>[_via_<data_source>][_<configurations>]</code>
--

各部分说明:

`ctl_`: 所有库函数使用的固定前缀。

`<action>`: 描述函数的主要操作。

- `init`: 用于模块的初始化过程，通常只在启动时调用一次。
- `step`: 用于执行模块在每个控制周期的计算，是实时循环 (如 ISR) 中的核心函数。
- `get, set`: 用于获取或设置模块的内部状态或参数。
- `input, output`: 用于标记在控制流程的特定阶段 (输入/输出) 调用的函数。

`<target_object>`: 指明函数操作的目标模块，例如 `pid`, `ramp`, `sogi` 等。

`_via_<data_source>`: (可选) 用于指定函数执行所依赖的数据来源或方式，例如 `_via_amp_freq` 表示通过幅值和频率参数进行设置。

`_<configurations>`: (可选) 用于区分同一模块的不同实现或配置，例如 `_par` 表示并联拓扑，`_2` 表示第二种实现方式。

示例:

`ctl_init_pid`, `ctl_set_pid_limit`: 来自 `pid.h` 的函数。

`ctl_init_ramp_generator_via_freq`, `ctl_step_ramp_generator`: 来自 `signal_generator.h` 的函数。

各个控制模块的结构都相似，以 PID 控制器为例。

<code>gmp_pro\ctl\component\intrinsic\continuous\continuous_pid.h</code>
--

<pre>/*-----*/ /* Standard PID Controller */ /*-----*/</pre>
--

```

/**
 * @brief Data structure for a standard parallel or series PID controller.
 */
typedef struct _tag_pid_regular_t
{
    // Parameters
    ctrl_gt kp; //!< Proportional gain coefficient.
    ctrl_gt ki; //!< Integral gain coefficient.
    ctrl_gt kd; //!< Derivative gain coefficient.

    // Limits
    ctrl_gt out_max; //!< Maximum output limit.
    ctrl_gt out_min; //!< Minimum output limit.
    ctrl_gt integral_max; //!< Maximum integrator limit.
    ctrl_gt integral_min; //!< Minimum integrator limit.

    // State variables
    ctrl_gt p_term; //!< Current P term for anti-windup
    ctrl_gt i_term; //!< Current I term.
    ctrl_gt d_term; //!< Current d term for anti-windup

    ctrl_gt out; //!< The current controller output.
    //ctrl_gt sn; //!< The integrator sum.
    ctrl_gt dn; //!< The previous input value for the derivative term.
} ctl_pid_t;

/**
 * @brief Initializes a parallel-form PID controller.
 * @details Calculates  $k_i = K_p * K_i * T$  and  $k_d = K_p * K_d / T$ .
 * @param[out] hpid Pointer to the PID controller instance.
 * @param[in] kp Proportional gain.
 * @param[in] Ti Integral gain (Hz).
 * @param[in] Td Derivative gain (seconds).
 * @param[in] fs Sampling frequency (Hz).
 */
void ctl_init_pid(ctl_pid_t* hpid, parameter_gt kp, parameter_gt ki, parameter_gt kd,
parameter_gt fs);

/**
 * @brief Executes one step of the parallel-form PID controller.
 * @details Output =  $K_p * e(n) + K_i * \text{sum}(e(n)) + K_d * (e(n) - e(n-1))$ .
 * @param[in,out] hpid Pointer to the PID controller instance.
 * @param[in] input The current input error,  $e(n)$ .
 * @return ctrl_gt The calculated controller output.

```

```

*/
GMP_STATIC_INLINE ctrl_gt ctl_step_pid_par(ctrl_pid_t* hpid, ctrl_gt input)
{
    // update p term
    hpid->p_term = ctl_mul(input, hpid->kp);

    // Update integrator sum with anti-windup
    hpid->i_term = ctl_sat(hpid->i_term + ctl_mul(input, hpid->ki), hpid->integral_max,
        hpid->integral_min);

    // update d term
    hpid->d_term = ctl_mul((input - hpid->dn), hpid->kd);

    // Output = P_term + I_term + D_term
    hpid->out = hpid->p_term + hpid->i_term + hpid->d_term;

    // Saturate final output
    hpid->out = ctl_sat(hpid->out, hpid->out_max, hpid->out_min);

    // Store current input for next derivative calculation
    hpid->dn = input;

    return hpid->out;
}

/**
 * @brief Executes one step of the series-form PID controller.
 * @details Output =  $K_p * [e(n) + 1/T_i * \sum(e(n)) + T_d * (e(n) - e(n-1))]$ .
 * This is approximated by applying  $K_p$  to all terms.
 * @param[in,out] hpid Pointer to the PID controller instance.
 * @param[in] input The current input error,  $e(n)$ .
 * @return ctrl_gt The calculated controller output.
 */
GMP_STATIC_INLINE ctrl_gt ctl_step_pid_ser(ctrl_pid_t* hpid, ctrl_gt input)
{
    // update p term
    hpid->p_term = ctl_mul(input, hpid->kp);

    // Update integrator sum based on the proportional output
    hpid->i_term = ctl_sat(hpid->i_term + ctl_mul(hpid->p_term, hpid->ki), hpid->integral_max,
        hpid->integral_min);

    // update d term
    hpid->d_term = ctl_mul((input - hpid->dn), hpid->kd);

```

```

// Output = P_term + I_term + D_term
// Note: In a true series form, Kd would also be scaled by Kp.
hpid->out = hpid->p_term + hpid->i_term + hpid->d_term;

// Saturate final output
hpid->out = ctl_sat(hpid->out, hpid->out_max, hpid->out_min);

// Store current input for next derivative calculation
hpid->dn = input;

return hpid->out;
}

/**
 * @brief Clears the internal states of the PID controller.
 * @param[out] hpid Pointer to the PID controller instance.
 */
GMP_STATIC_INLINE void ctl_clear_pid(ctl_pid_t* hpid)
{
    hpid->dn = 0;

    hpid->p_term = 0;
    hpid->i_term = 0;
    hpid->d_term = 0;

    hpid->out = 0;
}

/**
 * @brief Set PID controller PID limit
 * @param[out] hpid Pointer to the PID controller instance.
 * @param[in] limit_min PID output minimum.
 * @param[in] limit_max PID output maximum.
 */
GMP_STATIC_INLINE void ctl_set_pid_limit(ctl_pid_t* hpid, ctrl_gt limit_max, ctrl_gt
limit_min)
{
    hpid->out_max = limit_max;
    hpid->out_min = limit_min;
}

/**
 * @brief Get PID controller PID output item

```

```

* @param[in] hpid Pointer to the PID controller instance.
* @return ctrl_gt The calculated controller output.
*/
GMP_STATIC_INLINE ctrl_gt ctl_get_pid_output(ctl_pid_t* hpid)
{
return hpid->out;
}

```

1.3 平台提供的几项关键例程

GMP 提供了丰富的电机控制器和电力电子电源控制器例程，在实践过程中可以直接使用、或者方便地移植到特定平台

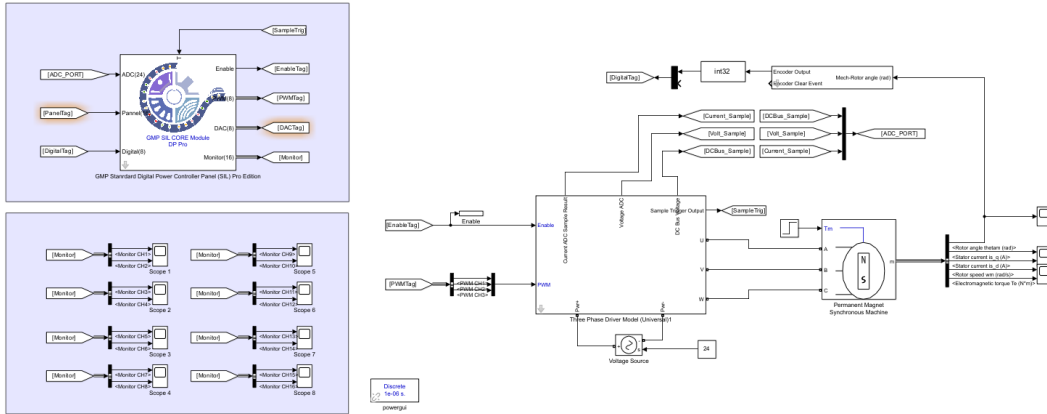
名称	工程路径（相对 GMP 仓库根目录）
关键说明	
FSBB	gmp_pro\ctl\suite\dps_fsbb
四开关 Buck-Boost 控制器，带线性化调制器	
SINV	gmp_pro\ctl\suite\pgs_sinv_rc
单相全桥双向变流器控制器，带重复控制改善波形质量	
INV_GFL	gmp_pro\ctl\suite\pgs_inv_GFL_inverter
三相并网变流器控制器，功率闭环，带 DSOGI-PLL 锁相环	
PMSM	gmp_pro\ctl\suite\mcs_pmsm_nt
永磁同步电机控制器，位置闭环，带 eSMO 观测器，和弱磁控制器	
ACM	gmp_pro\ctl\suite\mcs_acm
异步电机控制器，带异步电机角度计算器和磁链观测器	

2 GMP 提供的仿真资源

GMP 平台提供了同代码仿真机制，允许用户使用与运行在控制器中相同的代码联合 MATLAB Simulink 的硬件仿真，从而仿真、验证控制器的正确性。

在 simulate 工程中提供了电机控制器的主电路拓扑和电机，如下图所示。

通过 Visual Studio 提供的仿真编译环境,可以与 implement 中的 ctl_main 和 user_main 联合仿真。



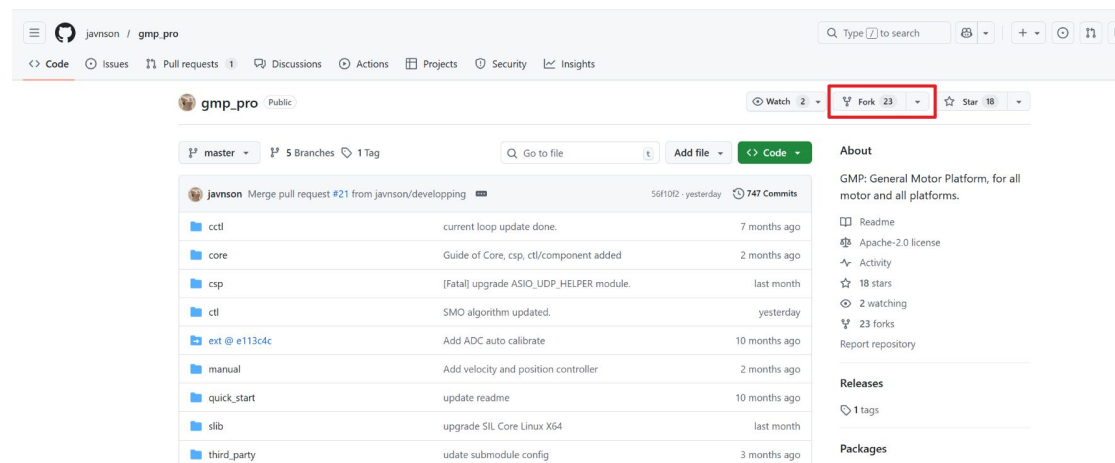
3 GMP 平台安装、部署说明

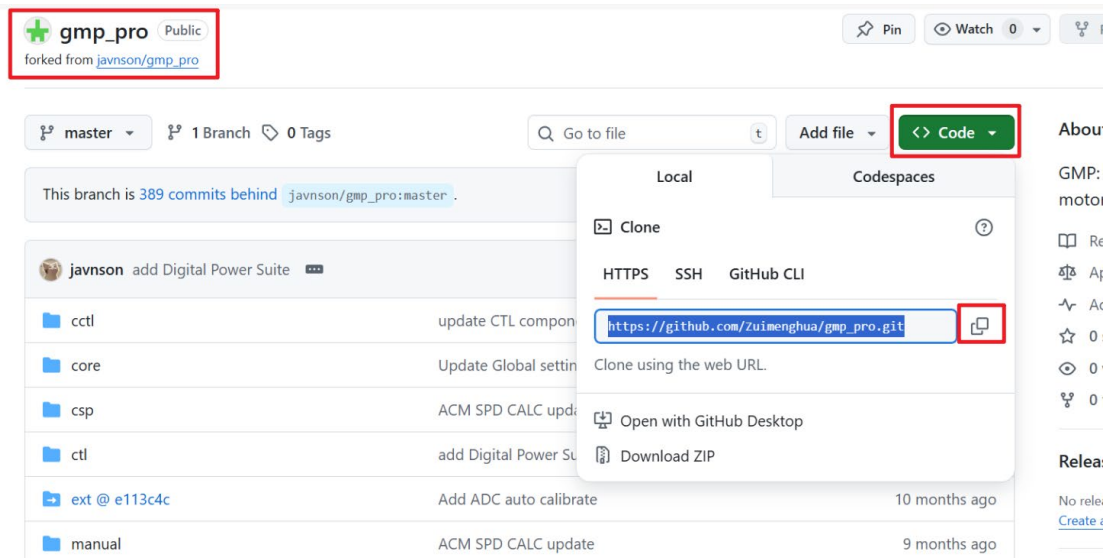
3.1 GMP 软件平台下载和安装

基本依赖：如果需要验证 python 是否安装，可以输入命令 `python --version`（注意此处是两个短划线 '-'），正确的情况下应该可以看到 python 的版本如 3.13.0。如果需要安装 python3，在 windows 环境中，在命令行中输入 `python3`，会弹出应用商店，直接在应用商店中安装即可。

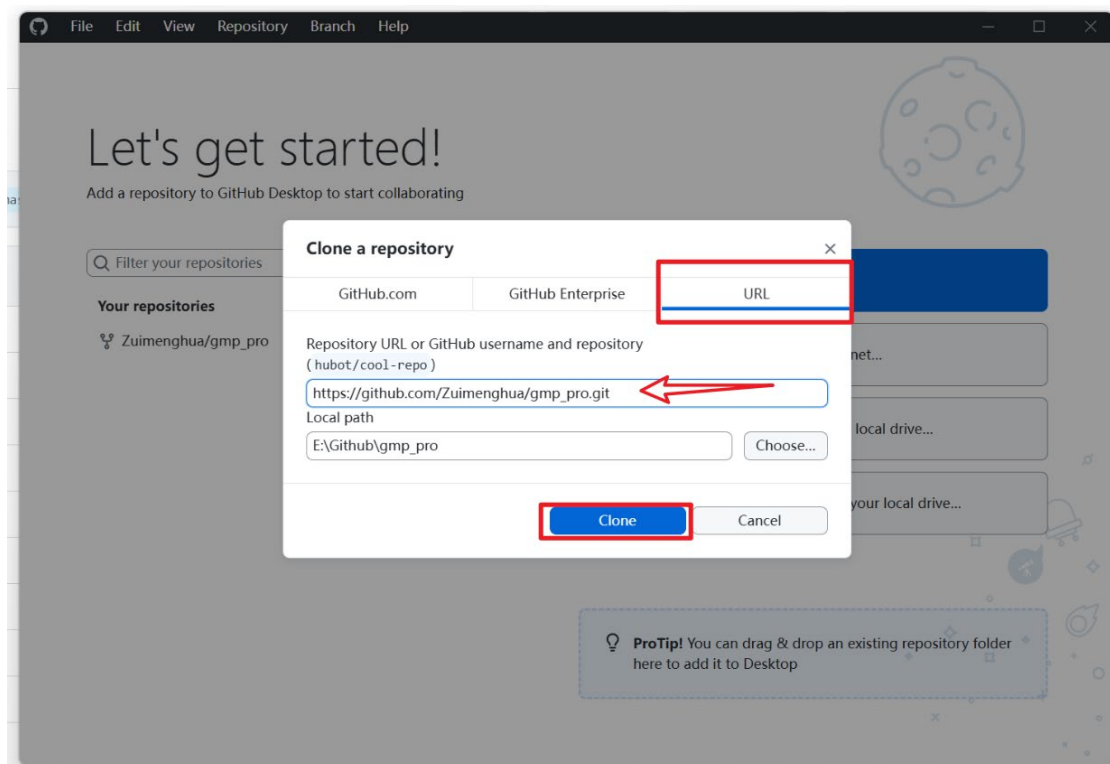
GMP 仓库：https://github.com/javnsn/gmp_pro，国内源请使用 gitee 仓库：https://gitee.com/zjuwumin/gmp_pro，下载后需要放在一个没有中文、没有空格的目录下。

下载建议使用 GitHub Desktop 软件（<https://desktop.github.com/download/>）管理，将仓库 fork 到自己的代码仓库中。

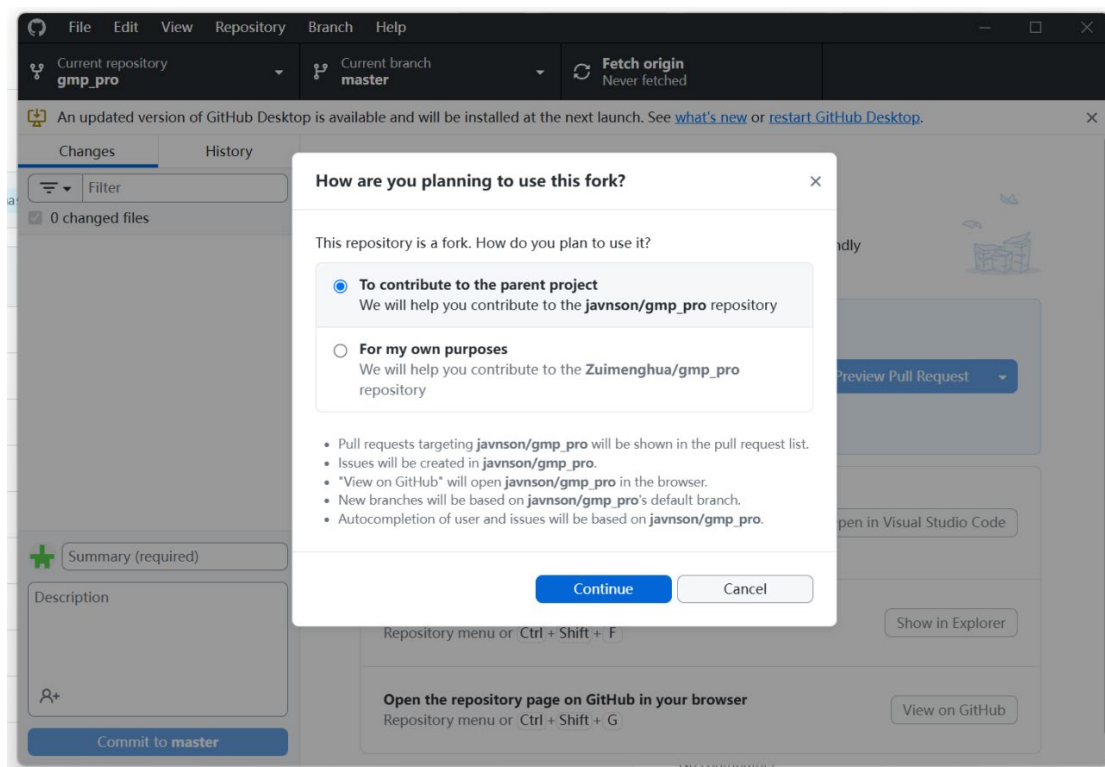




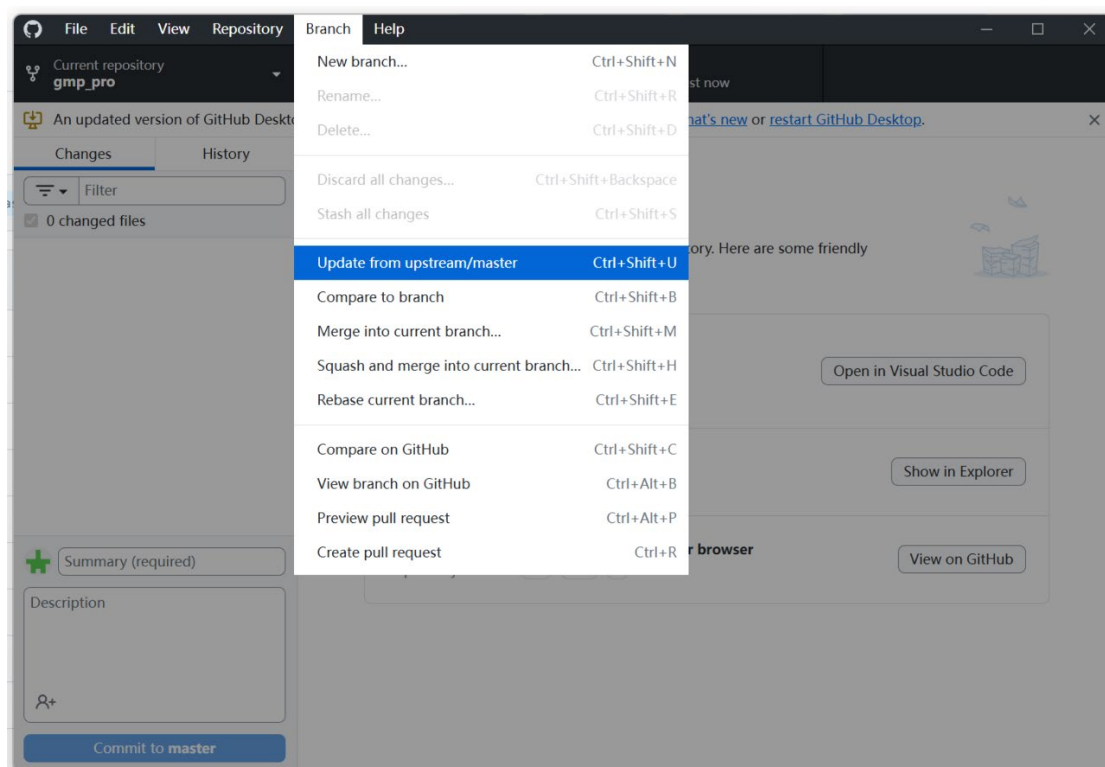
将 fork 后的地址复制到 GitHub Desktop 中下载：



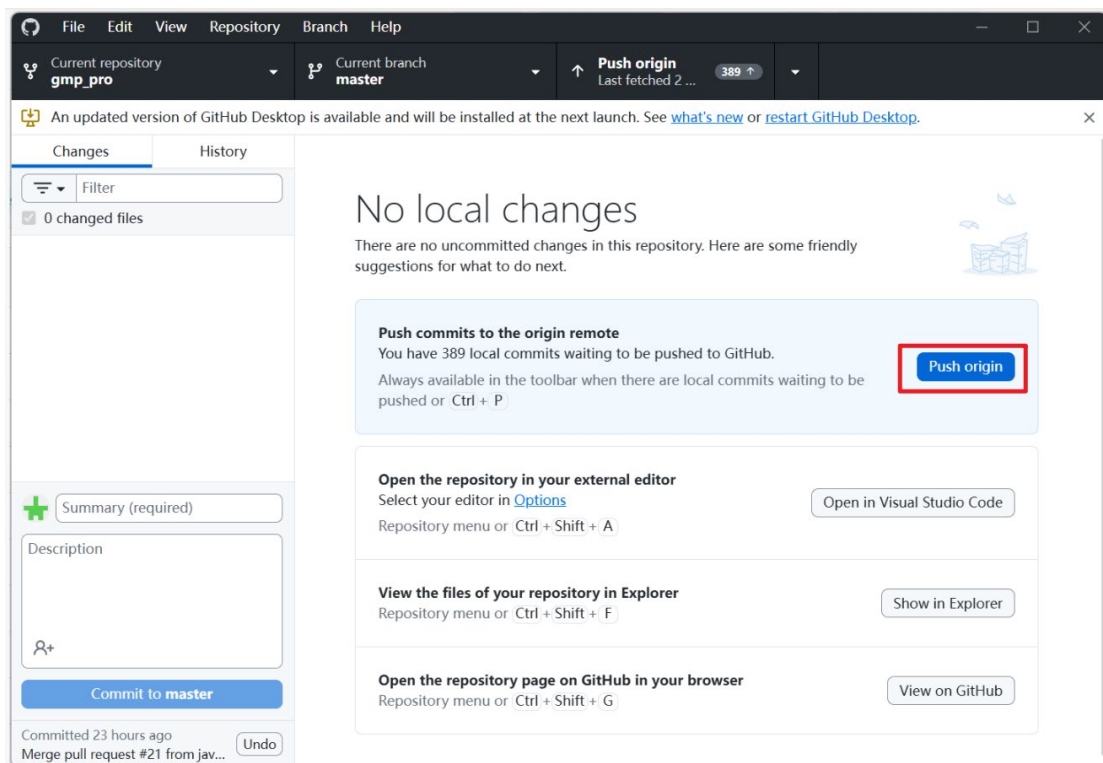
等待下载完成之后即可得到完整的 GMP 工具包。如下图所示，可以根据需要选择选项。



执行右上角 **Fetch origin** 命令后，可以如果 **GMP** 工程有更新，可以选择 **Branch-Update from upstream/master** 更新当前的分支到最新版本。



在有更新的情况下，会弹出 **Push Commit to origin remote** 选项，选择 **push origin** 后 **GMP** 包即可更新到最新版本。



选择 Show in Explorer 即可打开 GMP 所在根目录。运行根目录下的安装运行脚本 (install_gmp.bat, 不能使用管理员权限运行该文件, 否则会定位错误)。这个文件会基于 scoop 包管理器自动下载所有依赖, 出现下图说明正确安装。安装程序的主要工作是将 GMP 的路径注册一个环境变量, 方便程序找到代码的所在位置。同时通过 scoop 管理将来必要的一些支持库。

正确安装后可以在系统环境变量中找到名为 GMP_PRO_LOCATION 的用户环境变量, 这个环境变量的值指向 GMP 的根目录。后续软件的开发及头文件的查询将会基于此环境变量。

```
Scoop and vcpkg Auto-Install
Creating shim for 'osage'.
Creating shim for 'patchwork'.
Creating shim for 'sfdd'.
Creating shim for 'twopi'.
Running post_install script...done.
'graphviz' (14.1.2) was installed successfully!

[3/3] Integrating with Visual Studio
[ACTION] Performing user-wide integration
Applied user-wide integration for this vcpkg root.
CMake projects should use: "-DCMAKE_TOOLCHAIN_FILE=C:/Users/javnson/scoop/apps/vcpkg/current/scripts/buildsystems/vcpkg.cmake"

All MSBuild C++ projects can now #include any installed libraries. Linking will be handled automatically. Installing new libraries will make them instantly available.

[SUCCESS] Setup Complete

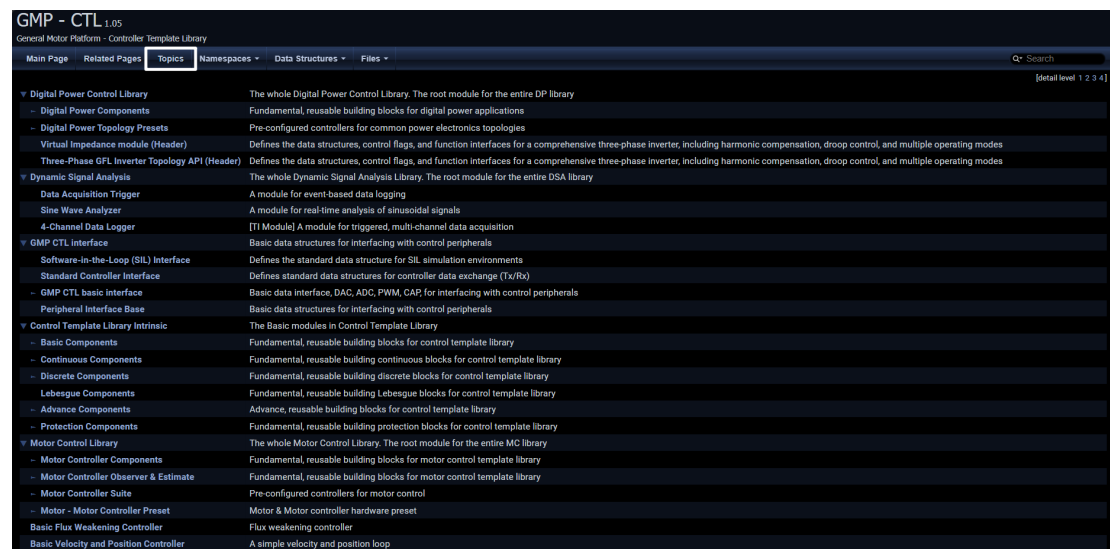
Python is installed.
Python Version:
Python 3.13.0
Environment variable GMP_PRO_LOCATION is: E:\lib\gmp_pro

[INFO] facilities: GMP-Core-C28x in ./is generating...
The file E:\lib\gmp_pro\tools\facilities_generator\json\facility_cfg.json has been generated successfully.
请按任意键继续...
```

在完成安装后, 可以在 gmp_pro/ctl 目录下运行 generate_doxygen.bat, 这个脚本将会编译并生成 ctl 的库帮助文档, 运行后看到 finished, 说明正确安装, 结果形如下图所示:

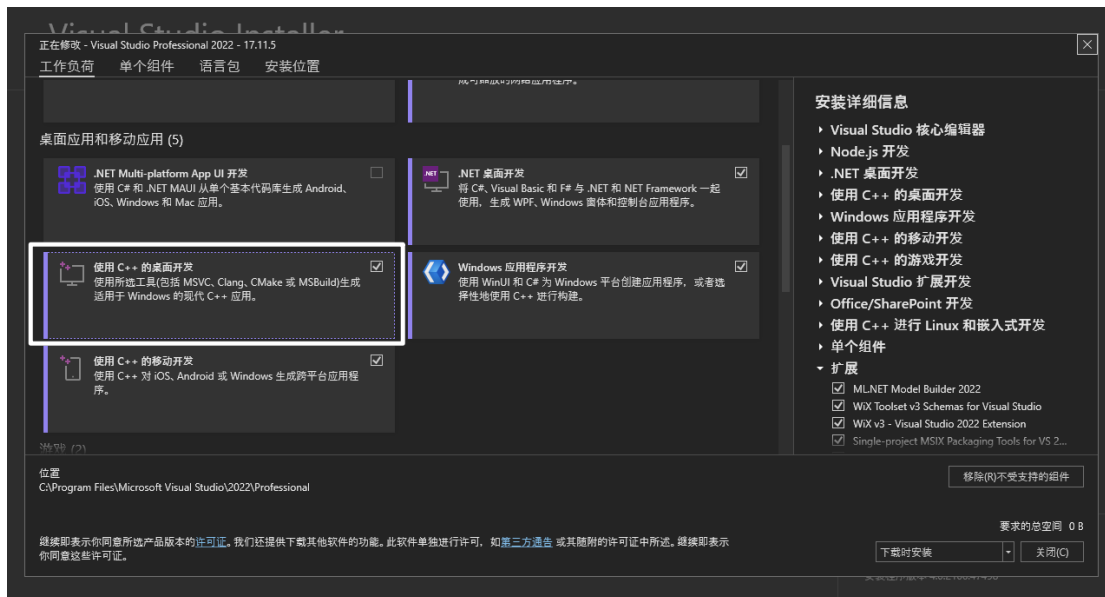
```
C:\Windows\system32\cmd.exe
Patching output file 495/520
Patching output file 496/520
Patching output file 497/520
Patching output file 498/520
Patching output file 499/520
Patching output file 500/520
Patching output file 501/520
Patching output file 502/520
Patching output file 503/520
Patching output file 504/520
Patching output file 505/520
Patching output file 506/520
Patching output file 507/520
Patching output file 508/520
Patching output file 509/520
Patching output file 510/520
Patching output file 511/520
Patching output file 512/520
Patching output file 513/520
Patching output file 514/520
Patching output file 515/520
Patching output file 516/520
Patching output file 517/520
Patching output file 518/520
Patching output file 519/520
Patching output file 520/520
type lookup cache used 9750/65536 hits=90525 misses=10552
symbol lookup cache used 14089/65536 hits=157905 misses=14055
finished...
请按任意键继续 . . .
```

编译 CTL 帮助文档后，可以在 `gmp_pro\ctl\doc\html` 目录下找到 `index.html`，使用浏览器打开，选择 `topic` 选项卡，即可看到 CTL 库提供的各个元件帮助文档。

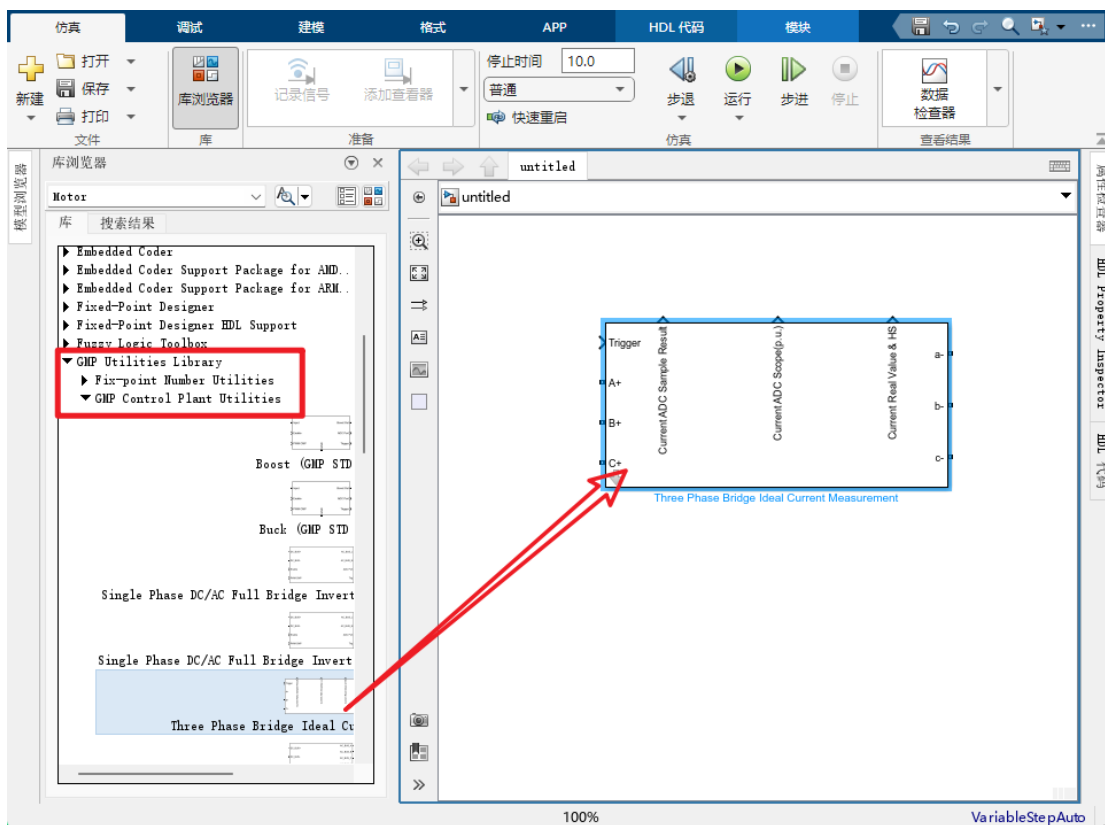


3.2 基于 Simulink 的仿真环境搭建

GMP 的仿真功能依赖于 MATLAB（运行主电路，大于 2022b 版本）和 Visual Studio（运行仿真控制器，需要安装基于 Windows 的 C++开发相关包）。



Simulink 支持库: gmp_pro/slib 文件夹下有 install_gmp_simulink_lib.m, 在 MATLAB 的目录切换到脚本所在目录后, 运行这个脚本即可安装 GMP 的 Simulink 支持库。正确安装后可以在 MATLAB-Simulink 中看到 GMP 的 Simulink 库。



如果有老版本的 GMP Simulink 库已经安装了, 请先运行老版本的 Uninstall 程序卸载 Simulink 库之后再运行新版本的安装程序, 否则因为文件名称冲突会导致安装失败。

如果 MATLAB 报错显示已经打开了同名的模型, 请彻底关闭 MATLAB 后重启, 并在启动后运行 uninstall_gmp_simulink_lib 脚本, 之后再重新运行安装脚本。

3.3 基于 CCS 的 C28x 系列 DSP 环境

安装 CCS12.8.1（不要安装 CCS20.x 版本，支持做得不是很好），使用 CCS12 版本即可。

<https://www.ti.com.cn/tool/zh-cn/download/CCSTUDIO/12.8.1>;

C2000ware 作为芯片支持包（建议使用 5.03 版本）

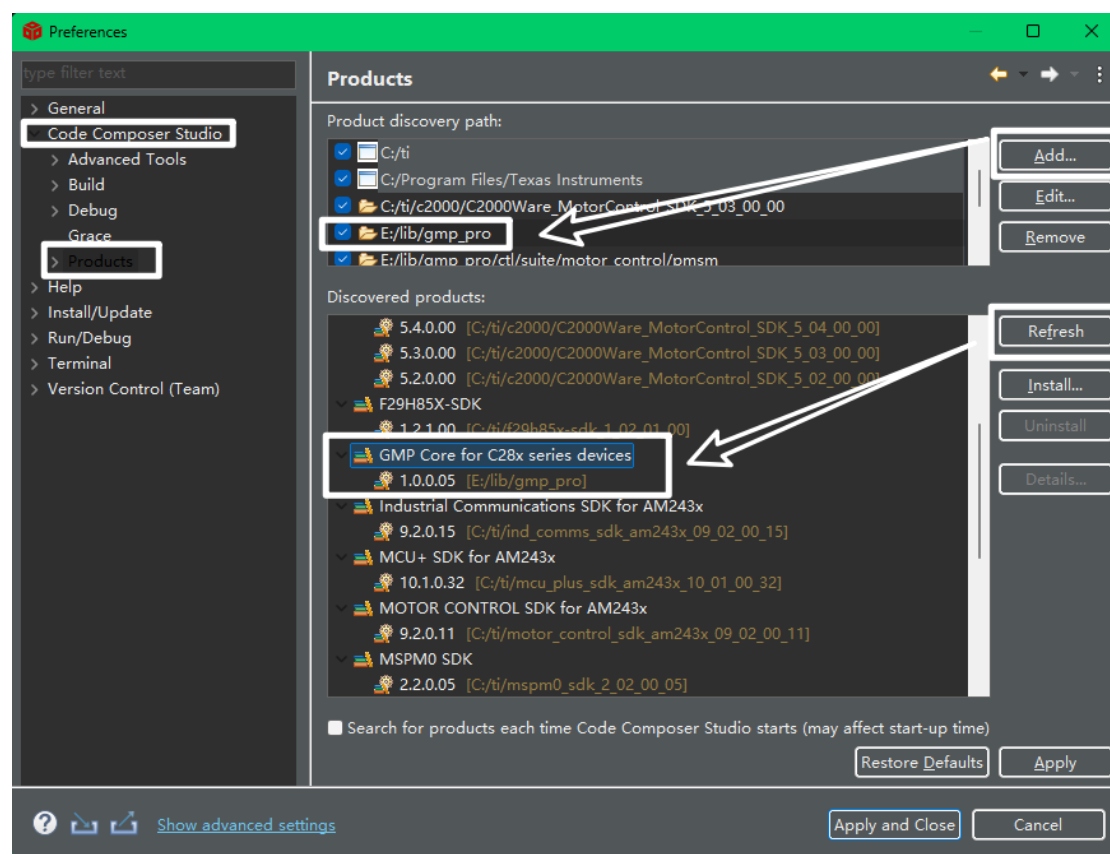
<https://www.ti.com.cn/tool/zh-cn/download/C2000WARE/5.03.00.00>;

Sysconfig 作为芯片资源管理

<https://www.ti.com.cn/tool/zh-cn/download/SYSCONFIG>。

将 GMP 作为 product 添加到 CCS 中，在 CCS 的 Window 菜单中选择 Preference 选项打开首选项窗口，在窗口中选择 Code Composer Studio 的 Products，将 GMP 库所在目录添加在目录中，刷新后即可看到发现的产品中有 GMP Core for C28x，如下图所示。

如果后续编译错误信息显示找不到 C2000Ware，或者特定版本的 Sysconfig 同样可以借助 Preference 选项卡。



3.4 基于 STM32 的开发环境安装

ST-STM32 环境：安装 Keil5 及相应芯片的支持 pack，CubeMX（<https://www.st.com.cn/zh/development-tools/stm32cubemx.html>）。

4 基于 Git 管理项目代码

我们要求大家一定使用 git 管理自己的项目，并且**每一个小组的每一个项目**形成一个独立的仓库，每完成一个功能的调试就完成至少上传一次变更，并且在 comment 中详细说明修改的关键内容，当前调试的状态。将 GMP 作为一个地基软件来使用。

用好 git 能够帮助大家在后续解决很多代码回溯、版本不统一的问题。

GMP 当前的项目可以脱离原始文件夹，直接拷贝出去作为一个独立的项目运行/存在/编译。比如 iris_280039c_board 这个工程，也比如 dps_fsbb 这个工程。

我们要求大家一定需要使用 Git 跟踪 GMP 的仓库，随着大家开展工作，我们将会根据大家的需求，以及大家的贡献，共同让这个仓库成长起来，让大家全副武装去考试。

4.1 Git 功能概述

Git 是一个分布式版本控制系统。它的主要作用是精确记录一个或多个文件的每一次历史更改状态。使用 Git 作为版本控制系统是现代编程系统的典型做法，其优势有：

告别“文件重命名”地狱：你不再需要在电脑里保存无数个凌乱的副本（如 main_v1.c, main_final.html）。Git 用一个清爽的文件夹就能在底层管理成千上万的历史版本，让你的工作区永远保持整洁。

终极的“后悔药”（时光倒流）：无论是误删了极其关键的代码，还是新加的功能导致整个系统彻底崩溃，Git 都能让你一键回退（Rollback）到过去任何一个正常运行的节点，赋予你无畏试错的底气。

精确定责与溯源：在团队合作中，Git 会精准记录下“每一行代码、在什么时间、由谁修改”（即强大的 git blame 功能）。这不仅是为了责任划分，更是为了在遇到复杂 Bug 时，能立刻找到最熟悉这段代码的负责人。

高效并行的团队协作：如果没有 Git，两个人同时修改同一个文件最后肯定会相互覆盖。Git 能够聪明地分析代码差异，让多名开发者在各自独立的空间（分支）里同时写代码，最后完美地合并在一起。

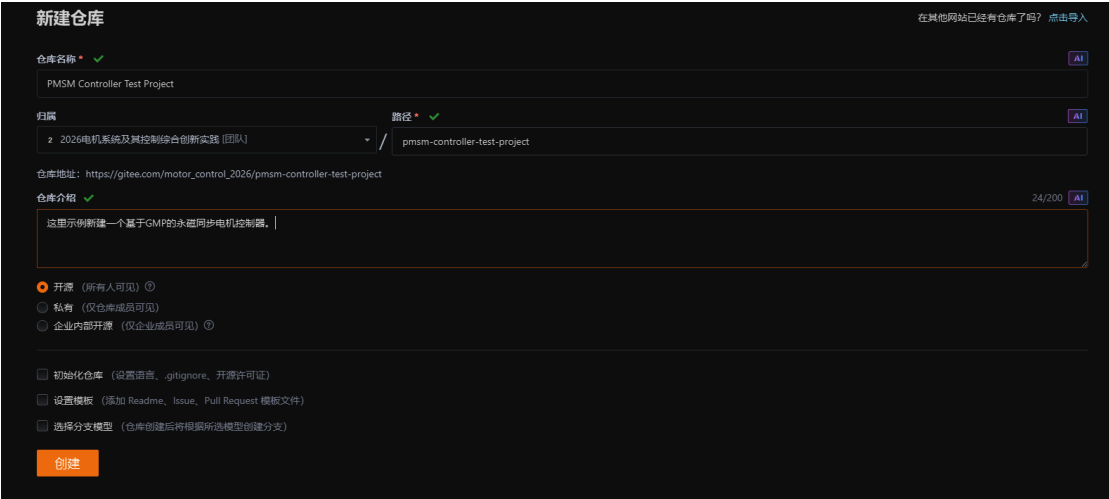
GMP 这一项目也是通过 Git 工具组织管理的，我们同样建议大家使用 Git 工具来管理 GMP 库代码和自己的工程代码。部署 GMP 提供的功能时建议使用 GMP pro 仓库提供的 release 版本，或者使用 fork 的方法直接下载 GMP 仓库的内容，之后以 Suite 工程的单位管理项目，不用带着“庞大的”GMP 到处复制。

4.2 以 Suite 为单位管理自己的工程项目

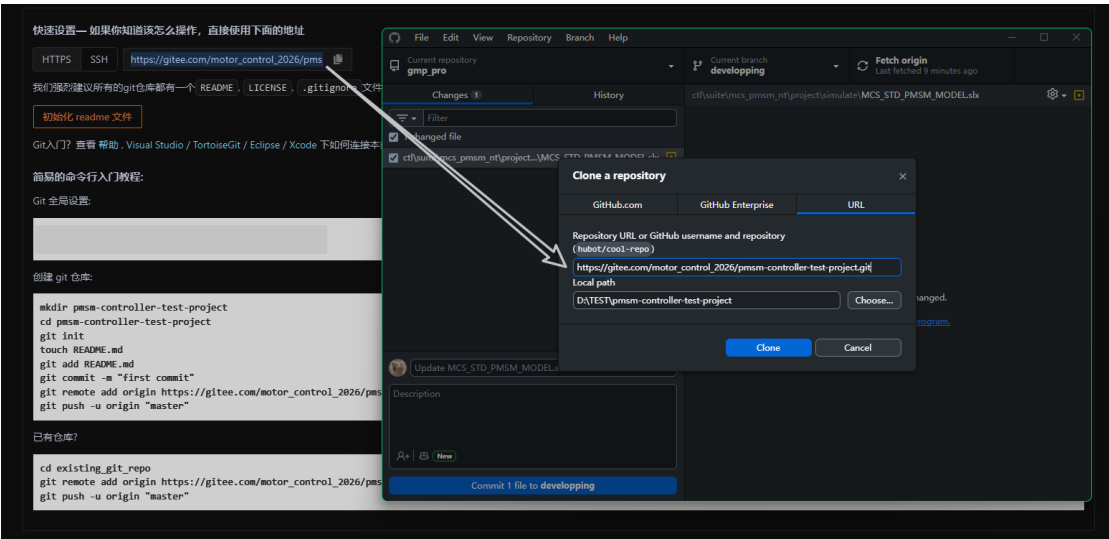
将需要拓展或使用的工程（比如：gmp_pro\ctl\suite\mcs_pmsm_nt），拷贝到特定的工程的目录（比如 D:\根目录下）。GMP 的头文件和源文件是通过系统环境变量（在安装时写入系统的）指定的，如果在本机移动 GMP 仓库的位置，只需要重新运行 GMP 根

目录下的安装程序即可。注意，这个目录路径仍然不能有中文和空格等特殊字符。然后基于此工程创建一个仓库，用来管理工程相关代码。推荐的方法是创建一个仓库，然后将工程内容移动进去，作为项目初始化，之后再进行开发。参考创建步骤如下：

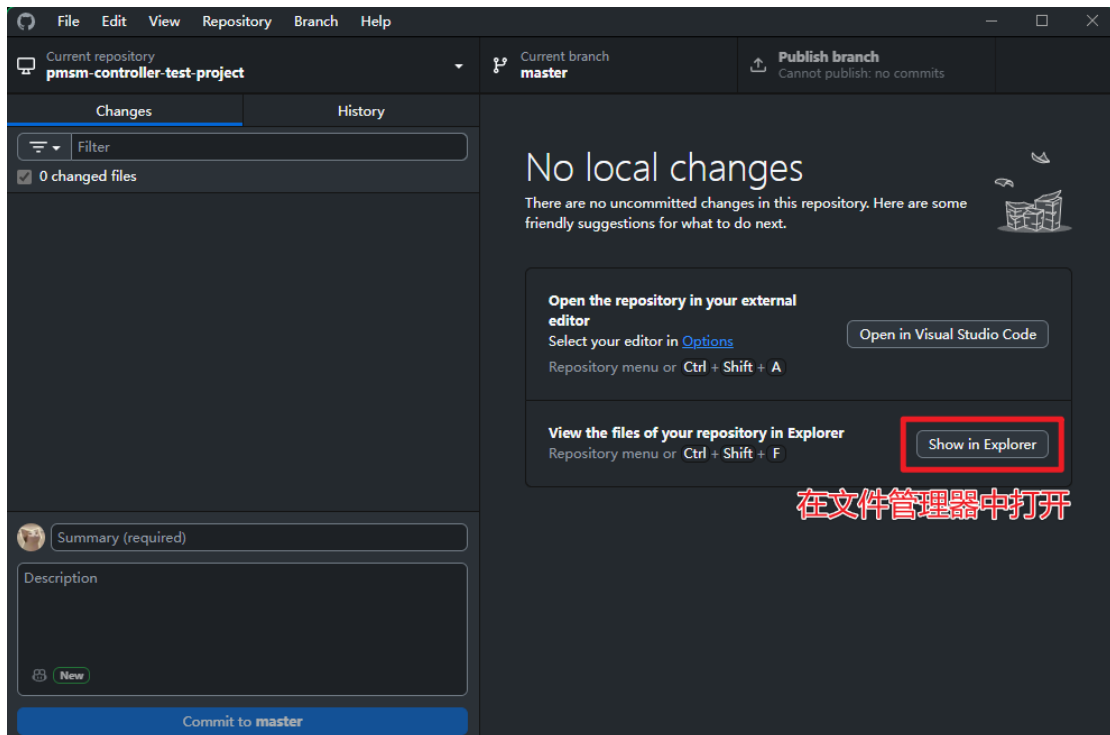
创建一个仓库，以 gitee 为例，填写仓库名称和仓库介绍，接下来，创建仓库即可。



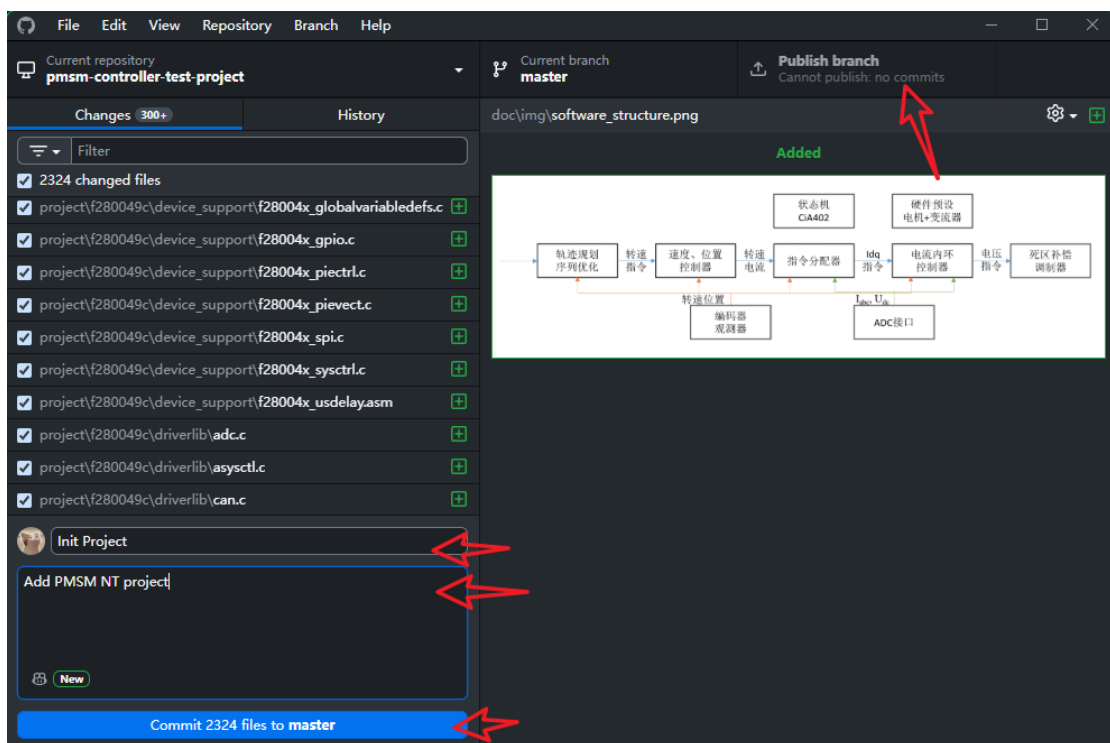
创建仓库后，可以使用 git 的相关工具将仓库克隆到本地，尽管仓库内容是空的，但是这样可以快速得到仓库的设置。



下载文件后，可以在文件资源管理器中打开，然后将项目内容粘贴到文件夹中，之后可以在管理界面看到修改内容。比如将 gmp_pro\ctl\suite\mcs_pmsm_nt 文件夹中全部内容都粘贴进新建的项目文件夹中。



在正确粘贴入文件后可以看到如下所示的界面，写入本次修改的摘要和说明，之后可以通过提交 Commit。接下来右上角的 Publish branch 将会点亮，此时可以点击 publish branch，此时将完成首次代码的提交。刷新仓库页面后可以看到项目代码的具体内容。



至此，完成了基于 Suite 的项目工程创建，可以尝试编译工程并运行。随后开始团队协作，完成电机控制的具体任务。对于当前电机需要特殊使用的模块，库中并没有提供的，可以在当前 suite project 工程目录下创建对应的头文件和源文件。

在工程文件夹中，比如 gmp_pro\ctl\suite\mcs_pmsm_nt\project\f280049c 目录下，有

`gmp_src_mgr` 文件夹，这个文件夹中有 `gmp_fac_generate_src.bat` 脚本，这个脚本负责基于当前文件夹中存在的配置文件（`gmp_facility_cfg.json`）将 GMP 库中的相关库的源文件搬运到当前目录下。初次打开工程，或者 GMP 库更新之后通常都需要手动运行此脚本以同步修改。

仿真环境（Visual Studio）的工程中，`gmp_fac_generate_src.bat` 脚本将会在每次生成之间自动被调用。所以不应当修改 `gmp_src` 文件夹中的任何源文件，而应该修改 GMP 库中的源文件(.c)。

4.3 基于 git 仓库的小组协作

1. 什么是代码仓库 (Repository)?

仓库（Repository，简称 Repo）本质上就是一个被 Git 施了“魔法”的文件夹。在这个文件夹里，所有文件（代码、图片、文档等）的增加、删除、修改，都会被 Git 自动追踪并记录下来。

本地仓库 (Local Repository): 存在你个人电脑上的魔法文件夹。即使没有网络，依然可以进行代码的保存和历史版本切换。

远程仓库 (Remote Repository): 托管在互联网服务器上（如 GitHub 网站）的仓库，主要用于多端备份和团队协作共享。

2. 运转机制：Git 的三大核心区域

你可以把代码提交流程想象成“超市购物与结账”的过程：

工作区 (Workspace)就是你在电脑上实际看到、正在编辑的那个普通项目文件夹。相当于超市的货架区。你在这里自由地闲逛，拿起商品（修改文件）。在这个阶段，Git 只是静静看着你，并不会强制记录你的每一个小动作。

暂存区 (Staging Area / Stage / Index)是一个隐形的过渡区域，用于临时存放你准备在下次提交时“打包”在一起的变动。相当于你的购物车。假设你今天在工作区修改了 5 个文件，其中 3 个是修复 Bug，另外 2 个是写了一半的新功能。在规范的开发中，你不应该把它们混在一起提交。此时，你可以精准挑选那 3 个修 Bug 的文件放进购物车，准备单独结账。暂存区给了你极大的灵活性去组织修改历史。

本地仓库 (Local Repository)是 Git 真正永久存储历史版本记录的地方（即隐藏的 `.git` 文件夹）。相当于超市的收银台账本。确认购物车里的商品没问题后，你推去结账。收银员会给你打印一张带有时间戳和小票号码（Commit ID）的回执。这笔订单从此被永久封存进账本，无法被轻易篡改。

3. 什么是分支？为什么需要分支？

你可以把分支想象成代码的平行宇宙。假设你正在开发一款游戏，主线代码存放在默认的 `main` 分支上（即线上正在稳定运行的版本）。

场景一：开发新功能。现在你想加入一个极其复杂的“飞行”功能，你怕写了一半代码跑不起来，影响到现有的主线。此时你可以基于 `main` 分支克隆出一个完全一样的平

行宇宙（比如叫 `feature-fly` 分支）。在这个宇宙里，你无论怎么折腾、报错，甚至把代码删光，都不会影响到 `main` 分支。等你完美实现了飞行功能，再把这个宇宙合并（Merge）回主宇宙中。

场景二：紧急修复 Bug。你正在开发新功能，老板突然说线上游戏有一个充值 Bug 必须立刻修复。此时你不需要丢弃手上写了一半的新功能代码，只需一键切换回 `main` 分支，创建一个 `bugfix-pay` 分支去修 Bug。修好合并后，再切回 `feature-fly` 分支继续开发你的新功能。

4. 常见的分支命名规范（业界通识）

在企业中，大家通常会约定俗成地命名分支，以便一眼看出分支的作用：

main 或 master：主分支。绝对稳定的代码，代表着线上正在运行的程序。绝对不要直接在这里写代码！

dev：开发分支。团队日常开发合并的主战场，用于集成测试。

feature/xxx：功能分支。例如 `feature/login`，用于开发登录功能。

bugfix/xxx 或 hotfix/xxx：修复分支。用于紧急修复 Bug。

4.4 几项关键的 git 操作

1. 核心本地工作流，记录你的代码脚印

在编写代码时，这三个命令将是每天使用频率最高的组合，它们构成了代码提交的完整闭环。

git status（状态雷达）：查看当前工作区和暂存区的状态。当不知道自己目前处于什么状态，或者下一步该干什么时，敲入 `git status` 准没错。它是新手的“定海神针”。

git add <file>（挑选与打包）：将工作区的修改（新建、修改或删除的文件）添加到暂存区。可以使用 `git add .` 一次性添加当前目录下所有变动。可以把它比喻成“把需要发货的商品放进纸箱”。

git commit -m "提交说明"（正式封箱）：将暂存区的内容正式提交到本地仓库，生成一个永久的记录（Commit ID）。比喻为“给纸箱贴上封条并写上发货单”。强调 `-m` 后面的说明必须清晰明了，让未来的自己和团队成员知道这次改动了什么。

2. 远程同步操作，与团队协作作战

当本地代码开发到一个阶段，或者需要获取队友的最新代码时，就需要和远程仓库（如 GitHub、GitLab）进行交互了。

git fetch（安全侦察）：从远程仓库下载最新的变动（包括提交记录、分支等）到本地，但绝不会自动修改你的当前工作区代码。这就像是“拿一份最新的快递清单来看看”，你可以借此检查队友做了哪些更新，确认安全后再决定是否合并到自己的代码中。它是一个非常安全的探索性操作。

git pull（一键同步）：从远程仓库获取最新版本，并直接合并到你当前的本地分支中。

核心公式: `git pull` $\approx$ `git fetch` + `git merge`。这是最常用的更新命令，相当于“不仅看了快递清单，还直接把包裹拆开和自己的东西混在了一起”。如果本地有未提交的修改，直接 `pull` 可能会产生冲突 (Conflict)，建议先 `commit` 本地代码再 `pull`。

`git push` (发布成果): 将本地仓库中已经 `commit` 的记录，推送到远程仓库中对应的分支。如果说 `pull` 是下载，那 `push` 就是上传。只有成功执行了 `push`，队友才能在远程仓库看到你的杰作。

5 基于 DSP280039C 的 IRIS Node 主控板说明

IRIS Node 由 GMP 仓库提供原生支持，参考工程代码地址如下，可以直接用 CCS 导入该工程。

```
gmp_pro\csp\c28x_syscfg\iris_280039c_board
```

5.1 DSP 的常用外设操作

本节以 DSP (280039C) 为例介绍 DSP 常用的外设操作:

5.1.1 PWM 发生

开关器件的操作，需要在 `output_callback` 中进行操作

用水平分辨率代替垂直分辨率，120MHz 主频，20kHz 的 PWM 频率，三角调制，所以默认水平分辨率为 3000，如果需要更高的开关频率则需要借助 HRPWM 的功能实现更高的水平分辨率。

关键函数:

通过调制函数可以将标么的结果映射到水平 PWM 的占空比上，通过写入比较器 A 来实现占空比的改变

```
EPWM_setCounterCompareValue(PHASE_BOOST_BASE, EPWM_COUNTER_COMPARE_A, ctl_get_fsbb_boost_cmp(&fsbb_mod));
```

如果需要同时改变频率做变频控制，则需要调用下面的函数改变 TB 的周期

```
EPWM_setTimeBasePeriod(EPWM1_BASE, new_prd);
```

5.1.2 DAC 输出

用来调试或者作为模拟的控制输出，需要在 `output_callback` 中进行操作，垂直分辨率 12bit。

关键函数:

让 DAC 直接输出一个标么的结果，这个数值应当均匀地映射到 DAC 的输出范围上

```
DAC_setShadowValue(IRIS_DACA_BASE, (uint16_t)(adc_v_out.control_port.value * 2048.0f + 2048.0f));
```

我要输出的数。针对6训练任务，ADC/DAC参考电压是3.3V（对应4096），1.65V对应的就是2048，1.65/2对应的就是1024*sin

5.1.3 ADC 输入

用来接收反馈和控制目标信号，需要在 `input_callback` 中进行操作，垂直分辨率 12bit。

关键函数：

直接获取 ADC 的数据，0-4095，12bit 的 ADC 结果：

```
ADC_readResult(FSBB_VIN_ADC_BASE, FSBB_VIN);
```

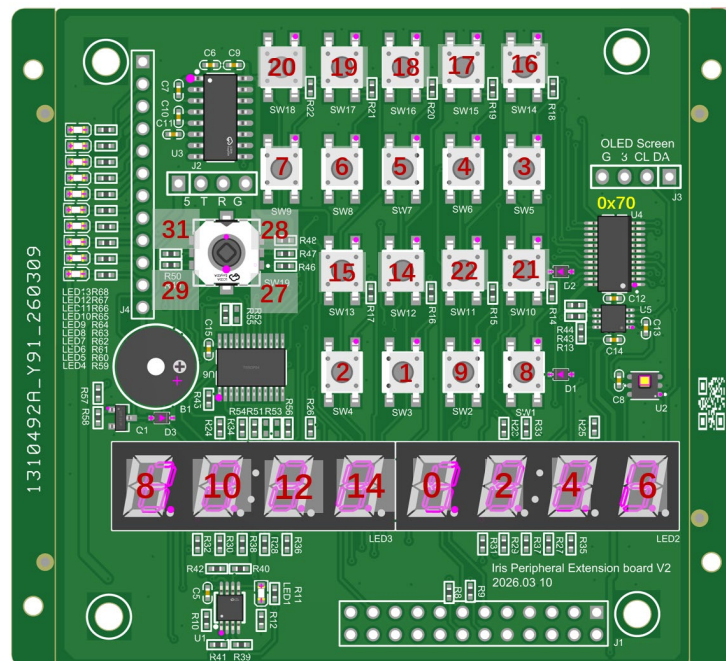
将获取的 ADC 结果送入 `adc_channel_t` 中得到一个标么结果：

```
ctl_step_adc_channel(&adc_v_in, ADC_readResult(FSBB_VIN_ADC_BASE, FSBB_VIN));
```

5.1.4 键盘和数码管的输入输出功能

键盘和数码管通过专用外设 HT16K33A-24SSOP 管理，通过 IIC 总线与 DSP 建立连接。

键盘用来作为用户交互输入，需要在 `user_loop` 中进行操作，用一个编号作为标识，除了键盘还有四向遥感，相应地也有不同的编码。



用来作为用户交互输出，需要在 `user_loop` 中进行操作，数码管的 ID 也有唯一标志，只不过已经被函数封装好了。

关键函数：

在一个非阻塞任务中询问按键管理芯片当前是否已经有按键被按下，如果需要响应按键消息，可以考虑也完成在此任务中。

并且默认情况下，按键的编号值会被数码管打印出来

```
gmp_task_status_t tsk_key_flush(gmp_task_t* tsk)
{
    ht16k33_dev_t* dev = (ht16k33_dev_t*)tsk->user_data;
```

```

    fast_gt key_id = 0;

    ec_gt ret = ht16k33_read_keys(dev, &key_id);

    // if meets error, close this task
    if (ret != GMP_EC_OK)
    {
        tsk->is_enabled = 0;
    }

    if (key_id != 0)
    {
        // response key message
        update_led_content_8byte(dev, led_lut[2], led_lut[0], led_lut[2],
led_lut[6], led_lut[20], led_lut[key_id / 10],
                                led_lut[key_id % 10], led_lut[20]);

        gmp_base_print("Receive Key Message, %d\r\n", key_id);
    }

    return GMP_TASK_DONE;
}

```

5.1.5 使用 OLED 液晶屏显示字符串

我们选用的 OLED 液晶屏尺寸为 1.3 寸, I2C 总线与 DSP 建立连接, 主控为 SSD1306。用来作为相对复杂的用户交互输出, 需要在 user_loop 中进行操作

关键函数:

在 OLED 初始化完成之后, 通过 sprintf 构造字符串, 并显示在 oled 的特定位置上, 第一个参数是水平的位置, 第二个参数是行数。

```

if (flag_init_cmpt == 1)
{
    sprintf(output_msg, "index: %d C", index++);
    oled_show_str(0, 2, output_msg);
}

```

5.1.6 旋转编码器（QEP）

用来作为电机或者位置编码的输入或用户交互的输入，对于控制的输入需要在 `input_callback` 中进行操作，对于用户交互的输入建议在 `user_loop` 中进行操作

关键函数：

读取编码器当前的数值

`EQEP_getPosition(EQEP_Encoder_BASE)`

5.1.7 UART 资源

F280039C 芯片有两个 SCI（UART）接口，一个用来调试，另一个用来作为通信接口。

在电力电子装备调试过程中，时常会遇到调试器异常中断连接，之后无法正常连接的情况，所以使用经过数据校验和保护串口通信就相对更加简单干练。在 `user_main` 中有一个任务 `tsk_dl_debug_device` 用来连接 DL(GMP 的 DataLink 模块)，对应的串口调试工具在

`tools\gmp_pil_server\gmp_debugger_v2`

运行 `run.bat` 即可运行调试工具。

默认情况下，可以在 `dict_m1` 中注册需要控制/调试/调整/编辑的非实时变量（刷新速度通常在 10Hz 以下的，这个模块可以基本上取代 CCS 或者 Keil 的 Watch 实时调试功能。

软件界面如下图所示，可以在 1 处从代码导入，将整个 `dict_m1` 变量的定义复制进去，示例如下代码。请注意，导入的代码应当和程序中的代码完全相同，否则会出现数据混乱。

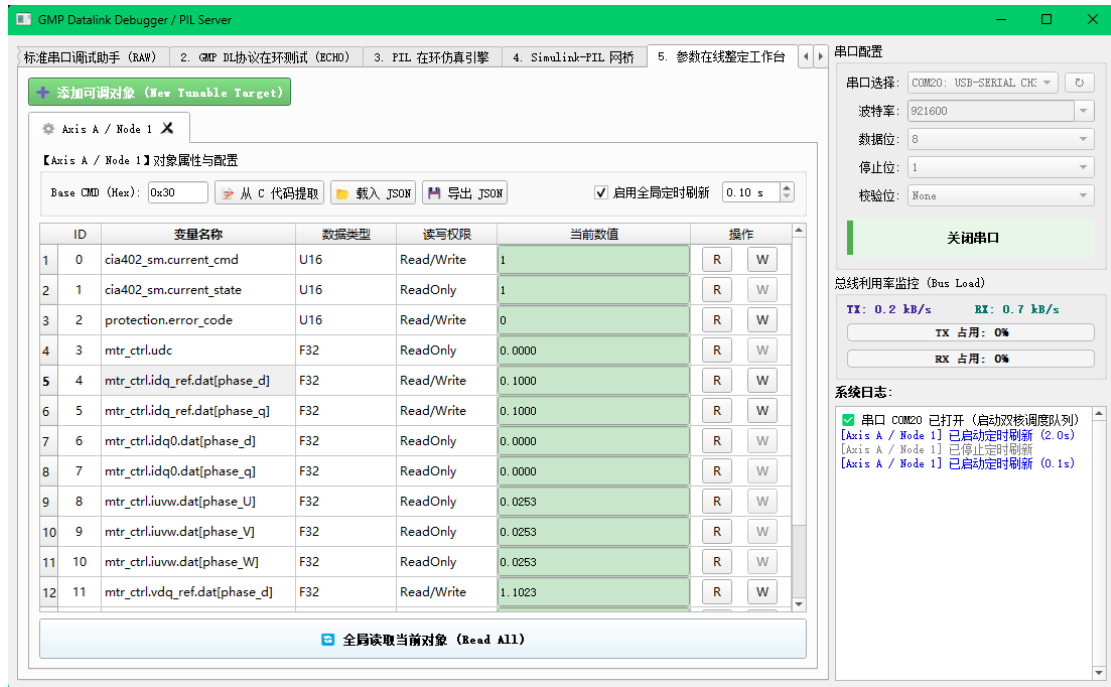
导入的代码示例

```
const gmp_param_item_t dict_m1[] = {
    {&cia402_sm.current_cmd, GMP_PARAM_TYPE_U16, GMP_PARAM_PERM_RW},
    {&cia402_sm.current_state, GMP_PARAM_TYPE_U16, GMP_PARAM_PERM_RO},
    {&protection.error_code, GMP_PARAM_TYPE_U16, GMP_PARAM_PERM_RW},
    {&mtr_ctrl.udc, GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RO},
    {&mtr_ctrl.idq_ref.dat[phase_d], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RW},
    {&mtr_ctrl.idq_ref.dat[phase_q], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RW},
    {&mtr_ctrl.idq0.dat[phase_d], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RO},
    {&mtr_ctrl.idq0.dat[phase_q], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RO},
    {&mtr_ctrl.iuvw.dat[phase_U], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RO},
    {&mtr_ctrl.iuvw.dat[phase_V], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RO},
    {&mtr_ctrl.iuvw.dat[phase_W], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RO},
    {&mtr_ctrl.vdq_ref.dat[phase_d], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RW},
```

```

    {&mtr_ctrl.vdq_ref.dat[phase_q], GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RW},
    {&mech_ctrl.target_velocity, GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RW},
    {&spd_enc.encif.speed, GMP_PARAM_TYPE_F32, GMP_PARAM_PERM_RO},
};

```



对于不满足 DL 通信协议的消息将会直接通过串口调试控制台输出，所有的通信串口数据可以在 DL 调试器的标准串口调试助手 RAW 中看到。通常这些消息通过 gmp 绑定的 debug printf 函数，也即可以通过下面的函数打印调试信息。

特别注意：调试信息打印函数不应当出现在任何控制过程（主中断）中，否则打印速度很慢，带来的阻塞将会导致控制器卡住跑飞。

打印调试信息

```
gmp_base_print(TEXT_STRING("Hello World!\r\n"));
```

使用 RS485（主板板载）或 RS232 通信协议（拓展版），可以使用类似下面的语句发送控制信号。如果需要接收信息，需要自己打开串口外设的接收中断，尽量避免阻塞接收。

打印到 RS485 或者 RS232

```
gmp_hal_uart_write(IRIS_UART_RS232, str, length, 10);
```

5.2 GMP 的数字控制器基本结构

数字控制器由“输入-控制-输出”结构构成。为了方便控制器代码与 MATLAB/Simulink 联合仿真，我们将代码分为跨平台的部分（user）和不跨平台的部分

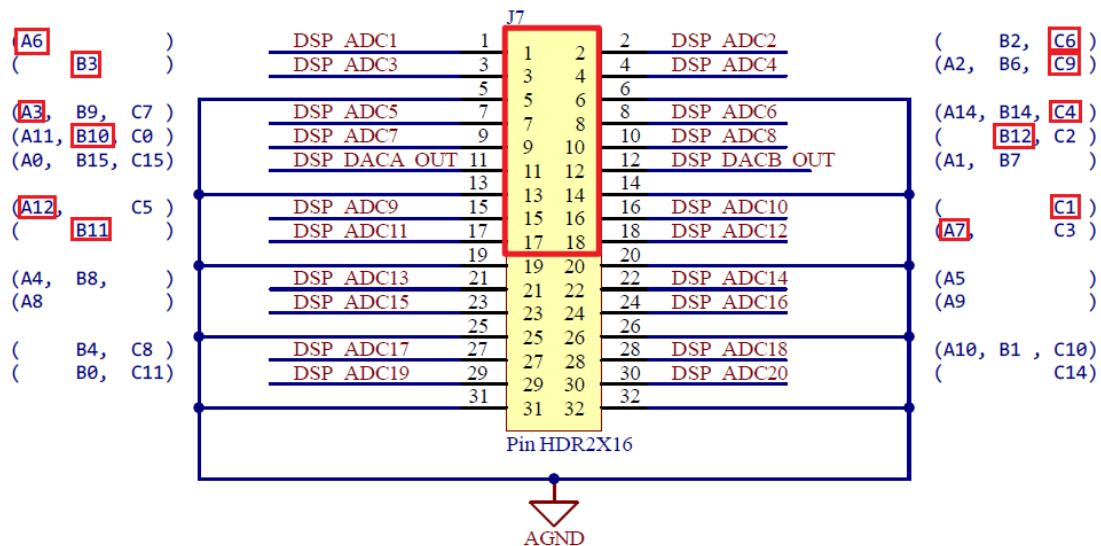
(xplt)。

直白的解释就是我们希望 user 的部分和平台无关，在与 MATLAB 联合仿真的时候仍然可以使用，所以这部分代码中必然不能包含外设相关的内容。如果一定需要外设操作，我们应当定义一个行为函数，这个特定行为的函数在 xplt 文件夹中的 c 文件中实现。

另一方面，对于 IRIS 的板子需要使用特定的一个 xplt，对于 MATLAB 仿真需要另一个不同的 xplt。

也就是 xplt 的目标是将 ADC，DAC，PWM 等外设的信息送到 CPU/MATLAB，而 user 文件夹中的内容应当包含最关键的用户逻辑和控制逻辑。

我们默认启用了 12 个 ADC 的通道，通常情况下这样的 ADC 配置足以支撑 50kHz 控制主频，12 个 ADC 的采样由 EPWM1(iris1，实际上对应的是 EPWM2 外设)触发，触发周期和 EPWM 的周期相同。



需要注意的是单个 ADC 外设完成读取模拟电压值需要经过采样和转换两个过程，这两个过程表征了转换速度。

$$T_{sample} = T_{SH} + T_{conv}$$

其中采样保持时间 T_{SH} 可以通过 Sysconfig 配置，默认设置到 150ns，转换时间 T_{conv} 对于 12bits 的 ADC 通常需要 10.5 个 ADCCLK 周期，即 175ns。相当于在同一个 ADC 外设下，两个通道之间存在 325ns 的延迟，通常情况下这并不是影响控制器性能的主要因素，且必要的时候可以补偿。

每一个 ADC 外设采样都是独立的，所以对于 280039C，可以同时采集 3 路 ADC 信号，这对于三相系统非常重要。

我们默认设置 EPWM 的频率是 20kHz，需要明白的是控制频率越高开关损耗越

大，通常情况下，这个控制频率已经足够使用（在 2kHz 以内基本上可以保证充分的控制，这也是我们控制器的目标）。

控制周期中应当规避任何的显示任务、初始化任务等。应当将初始化的部分、刷新控制参数的部分、显示的部分全部部署在主循环中。

GMP 仓库的 `ctl/component` 文件夹中提供了大量的控制器模块。比如 `biquad_filter.h` (`gmp_pro\ctl\component\intrinsic\discrete\biquad_filter.h`) 模块实现了非常强大的二阶滤波器，并且提供了各类不同应用场景的 IIR2 传递函数计算方法。

任何一个 component 都有三个必要的组成部分

1. `init` 函数，初始化函数，基本的命名规则是 `ctl_init_<component name>`，这个函数必须要在初始化过程中被调用，不能在控制过程中被调用。其中可以进行任何精度的浮点数计算，允许使用 `parameter_gt`。
2. `step` 函数，运行步进函数，基本的命名规则是 `ctl_step_<component name>`，这个函数将会在控制过程中被执行，其中只能使用 `ctl_gt` 类型，以及 `ctl_`打头的乘法、非线性函数等计算函数。
3. `clear` 函数，清除函数，基本的命名规则是 `ctl_clear_<component name>`，这个函数将会清除控制的状态，当控制器需要启动时，建议调用这个函数，让控制器回归零初始状态。

5.3 FPGA 模块主要功能说明

默认情况下，我们让 FPGA 作为 DSP 的外设模块，FPGA 为 DSP 扩展了 IO 资源、扩展了 ADC、DAC 资源。DSP 和 FPGA 之间可以通过 SPI 总线联系，同时提供了 ADC、PWM、时钟的同步信号，可以在 DSP 资源不够的情况下扩展各类模拟或者数字外设。

FPGA 是一颗合封芯片，其中包含了一个 2×32Mb 的 SDRAM，芯片等效为 23520 个 LE（查找表）资源。通常情况下一个 M0+ 的 ARM 内核需要消耗 3000 个左右的 FPGA 逻辑资源（大约 2000 个查找表，2000 个寄存器）大致这样的数量级。

此外，还应当熟悉 FPGA 的功能和 FPGA 拓展的模拟外设功能。在 IRIS 的板子上，FPGA 配置了性能相当强大的模拟外设，包括 16 位的 DAC 4 个通道，16 位的 ADC 8 个通道。结合 FPGA 提供的 29 个 18×18 的 DSP 模块，这种高分辨率、高性能外设对于信号、控制类题目是降维打击。

无论是多复杂的数字控制器(LTI、物理可实现)，总是可以被拆分成数个 IIR2 控制器级联，其传递函数为：

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

对应的时域表达式是：

$$y^{(n)} = b_0 x^{(n)} + b_1 x^{(n-1)} + b_2 x^{(n-2)} - a_1 y^{(n-1)} - a_2 y^{(n-2)}$$

通过 FPGA 提供的 DSP 和用 LUT 构造的乘法器，我们可以在 FPGA 中实现一个专门用于计算 IIR2 的核，FPGA 通常可以跑到 100MHz-200MHz 的频率，而采样到输出之间约有 100 个时钟周期，对于 FPGA 来说可以做非常丰富的任务（DSP 以及一些 MCU 做不到）。我们可以串联多个 IIR2 的核构造穿越频率极其陡峭，性能极其接近理想滤波器的数字滤波器。这是近些年数字信号处理的主流方向，即模数混合信号处理，用数字控制器实现模拟控制器几乎不可能实现的陡峭边沿。

默认情况下，我们将 FPGA 上的程序配置为 DSP 的从机，通过寄存器的方式允许 DSP 访问 FPGA（通过 SPI 总线）。在 iris_peripheral.c 文件中给出了具体的例子。

默认工程在 tools/iris_fpga_proj 目录下，是一个 TD Project，具体的寄存器映射信息可以参考工程下的 readme.md。

6 训练任务

1. 使用 DAC 通道 A 发生一个 100Hz 的正弦波，更新频率 20kHz，峰峰值为 1.65V，偏置为 1.65V。

注：发生正弦波需要使用 `ctl_sin()` 函数。

2. 使用 ADC 采集这个正弦波，并且减去 1.65V 偏置、以 1.65V/2 为单位 1 进行标么。 `interface adc_channel.h`

注：正弦波的标么可以使用 `adc_channel_t` 类型提供的功能。

参考头文件： `gmp_pro\ctl\component\interface\adc_channel.h`

3. 使用超前补偿器对于标么后的信号做超前补偿，超前补偿器的补偿频率设置为 100Hz，超前角度设置为 45° ($\pi/4$)，增益为 1。

注：超前补偿器可以使用 `ctl_lead_t` 模块。

参考头文件： `gmp_pro\ctl\component\intrinsic\discrete\lead_lag.h`

4. 将经过超前补偿得到的波形通过 DAC 通道 B 输出，更新频率仍然为 20kHz。

5. 将控制器输出的补偿后的波形标么在 [0, 1] 区间内，通过 PWM 调制发生在 IRIS_EPWM1 通道，死区设置为 500ns，死区极性设置为低电平（即 EPWM1A 和 EPWM1B 在死区时间同时输出低电平）。什么是死区，死区到底如何设置？

注：PWM 调制器的输出可以使用 `pwm_channel_t` 模块。

参考头文件： `gmp_pro\ctl\component\interface\pwm_channel.h`

6. 将控制器的超前角度通过数码管显示出来。

7. 允许通过按键设定超前控制器的超前角度。

注：超前控制器参数的更新需要在 `ctl_main` 中新增一个 `task`, 创建一个非阻塞任务，非实时解算。

8. 使用 OLED 将屏幕上显示超前角度和设定角度。

9. 使用 EQEP 旋转编码器作为输入，允许用户通过旋转编码设定目标超前相位。

10. 使用大彩屏将编码器输入和输出由大彩屏完成。

注：大彩屏使用 RS232 通信，需要从扩展板上 RS232 接口上连接。

7 主控板和扩展板的原理图

后文中只有最后两页是拓展板，其余全部内容为主控板的原理图。

A

A

B

B

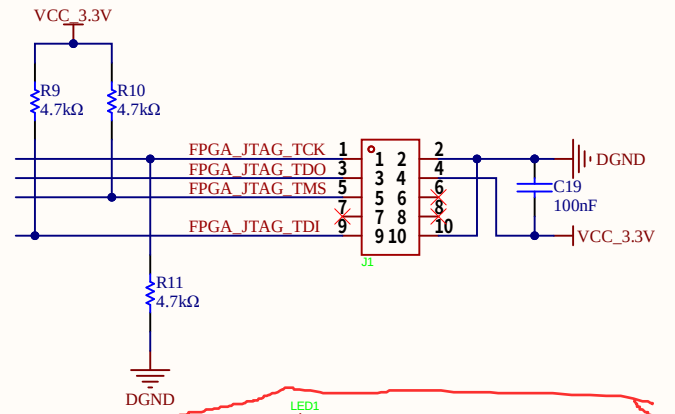
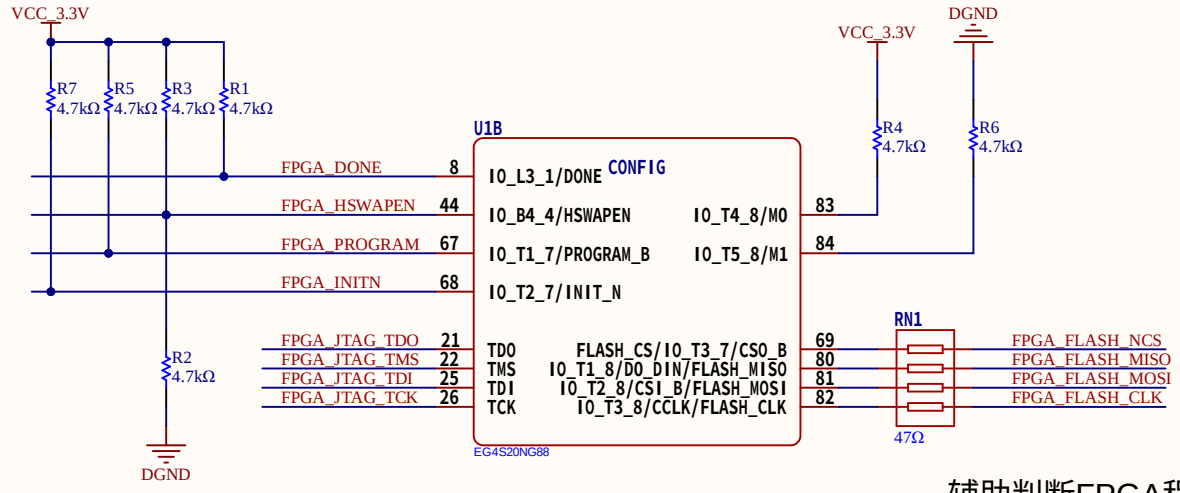
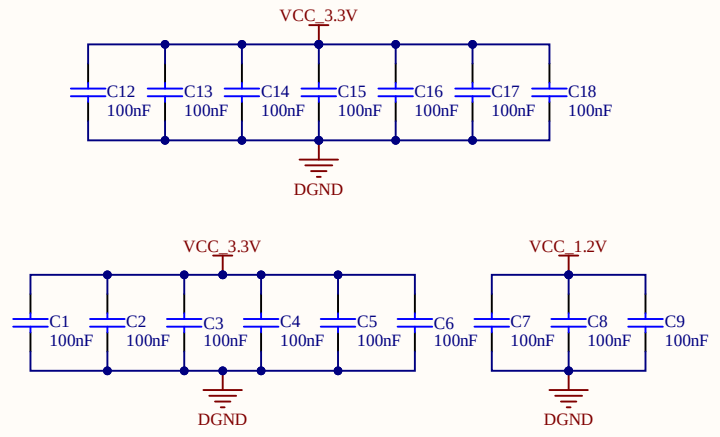
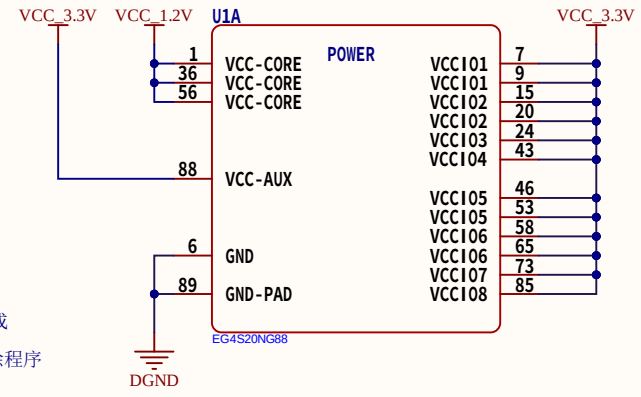
C

C

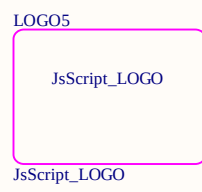
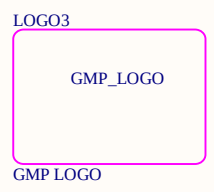
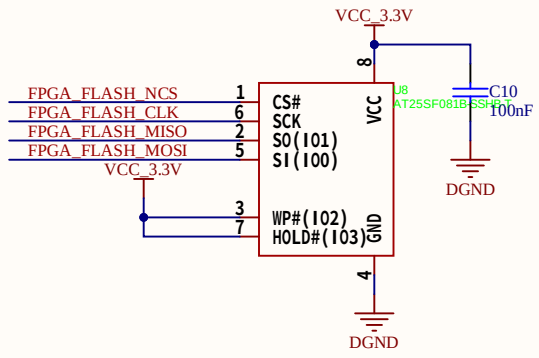
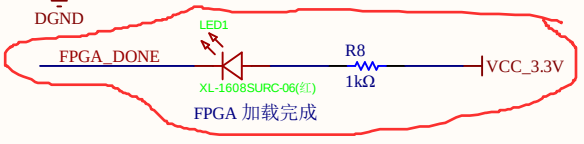
D

D

HSWAPEN IO 加载状态
DONE FPGA 配置完成
PROGRAM_B 芯片复位/清除程序



辅助判断FPGA程序是否成功烧录



Logo1
PCB
LOGO
WEEE logo

Logo2
PCB
LOGO
CAUTION. READ USER GUIDE BEFORE USE

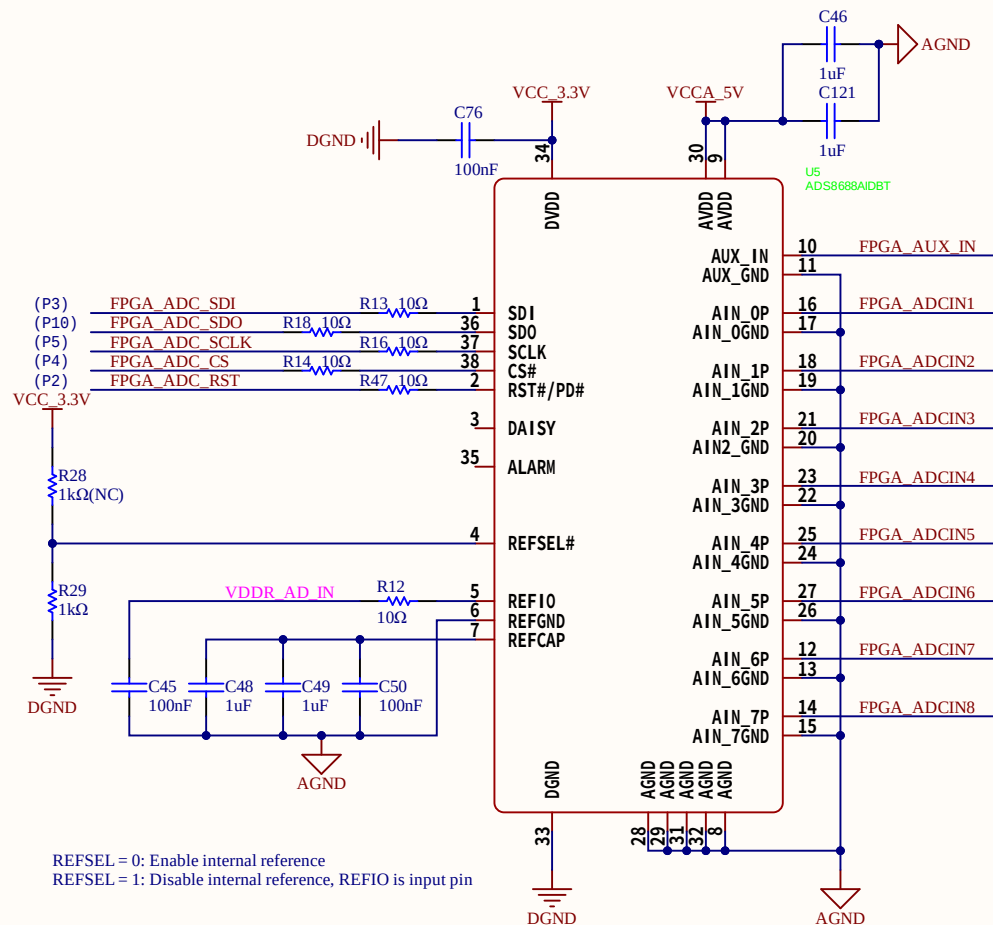
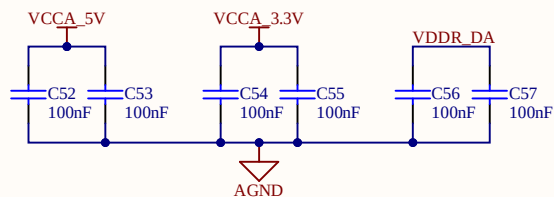
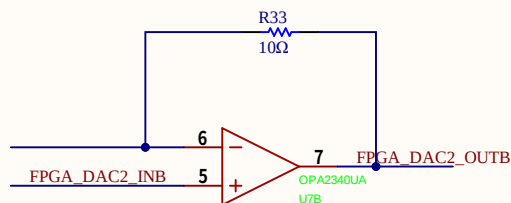
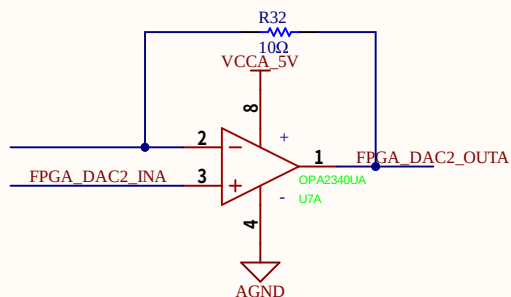
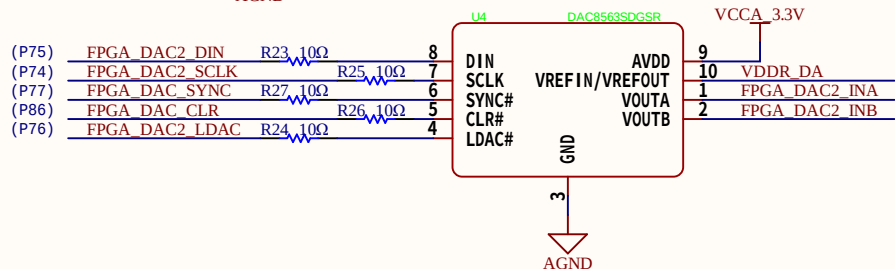
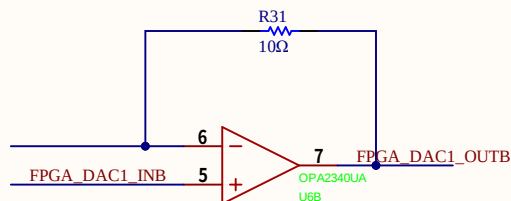
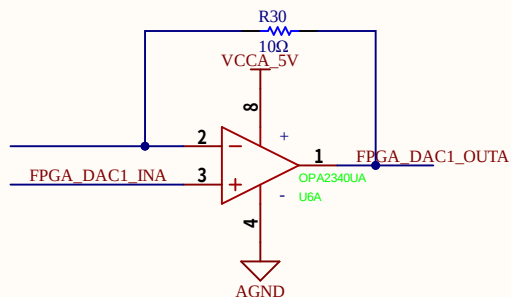
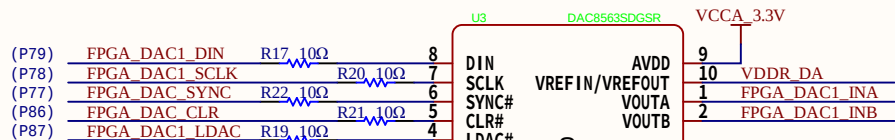
Title		
Size	Number	Revision
A4		
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\...FPGA_main.SchDoc	Drawn By:

1

2

3

4



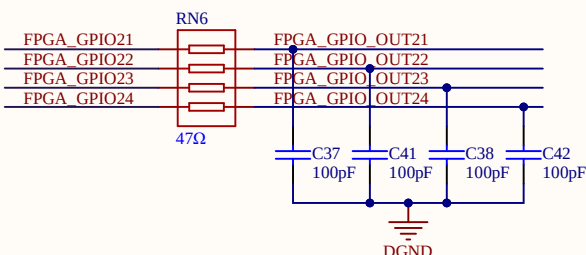
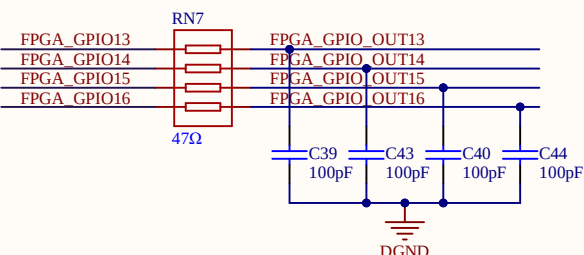
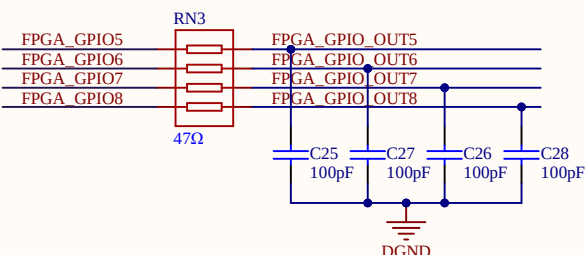
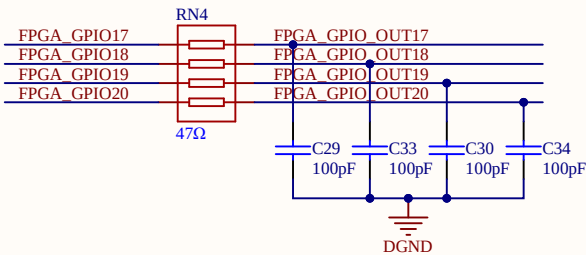
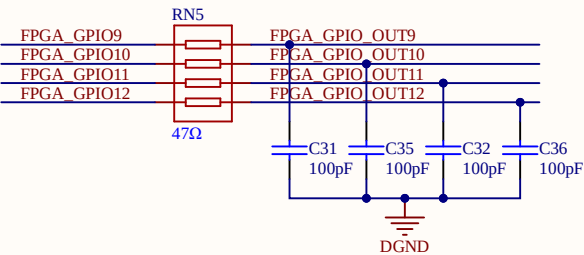
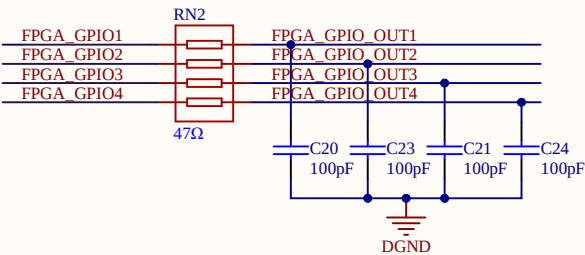
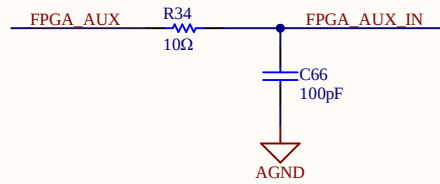
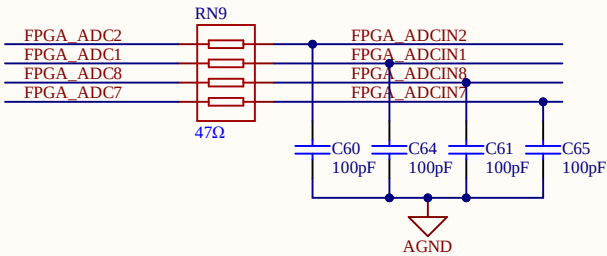
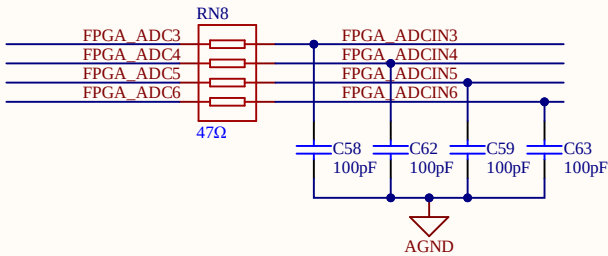
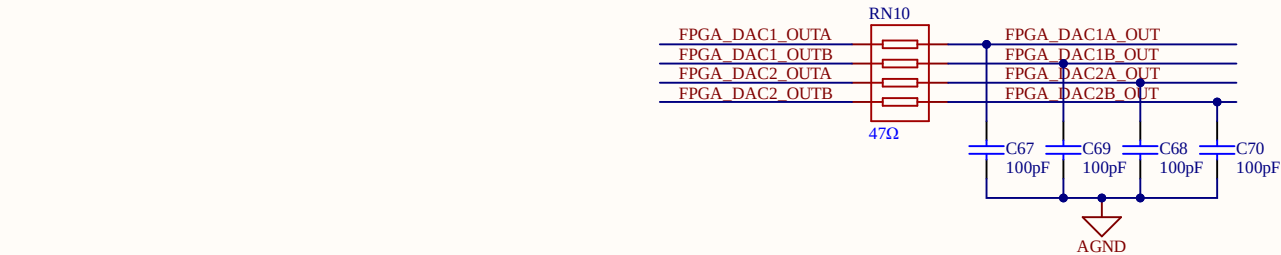
Title		
Size	Number	Revision
A4		
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\...\FPGA_Analog.SchDoc	Drawn By:

1

2

3

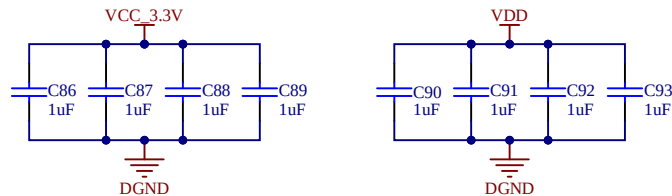
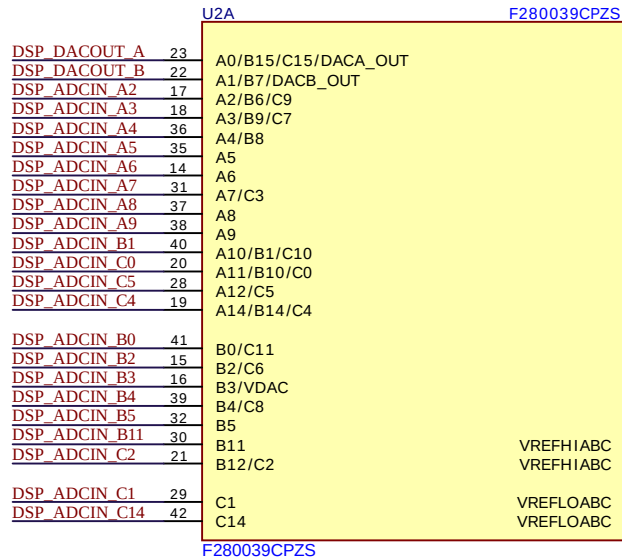
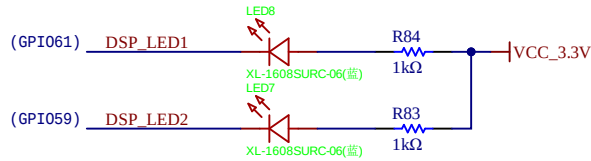
4



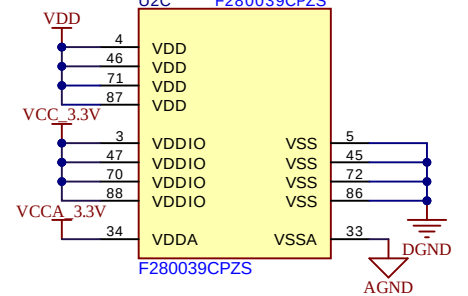
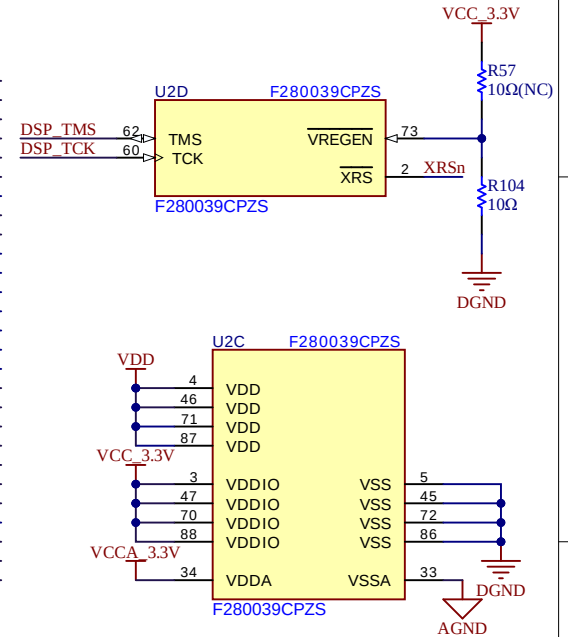
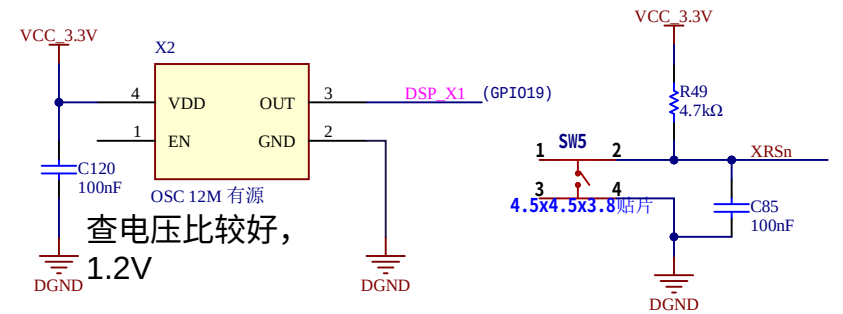
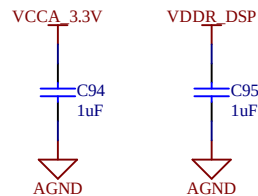
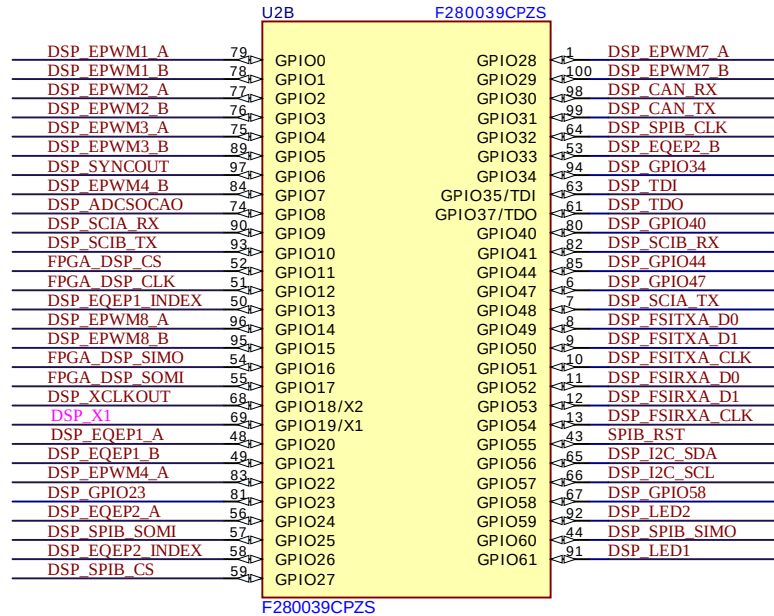
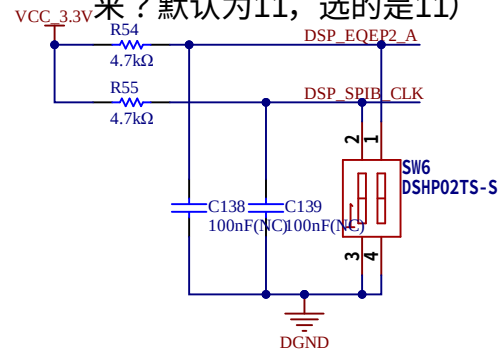
Title		
Size	Number	Revision
A4		
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\...\FPGA_Filter_Matrix_SchDoc	By:

Selected Boot Mode Chart

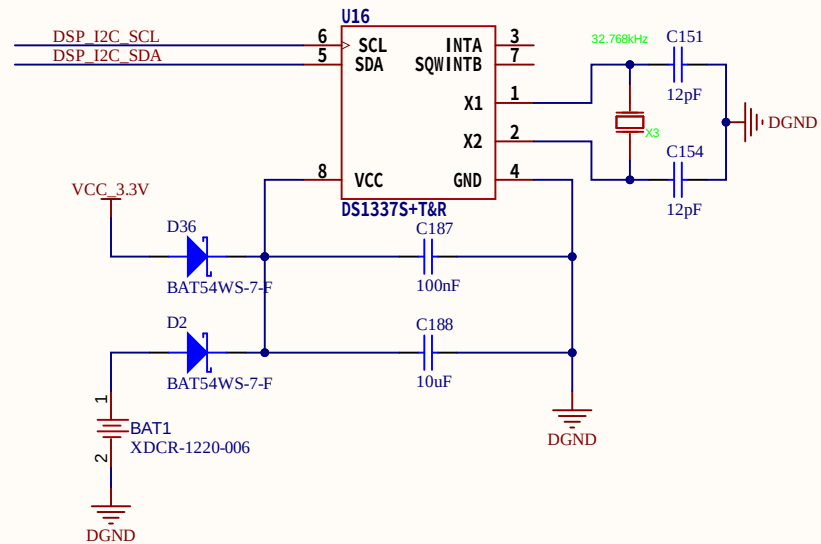
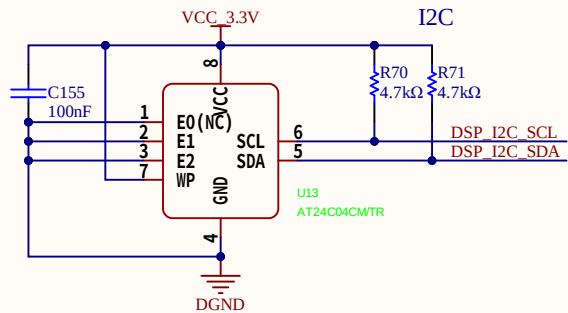
Mode #	GPIO24	GPIO32	Boot Mode
00	0	0	Boot from Parallel GPIO
01	0	1	Boot from SCI / Wait Mode
02	1	0	Boot from CAN
03	1	1	Boot from Flash



选择BOOT模式（启动源从哪里来？默认为11，选的是11）

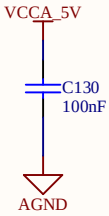
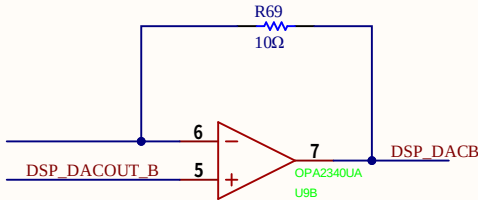
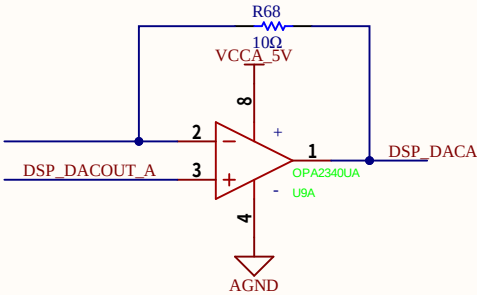
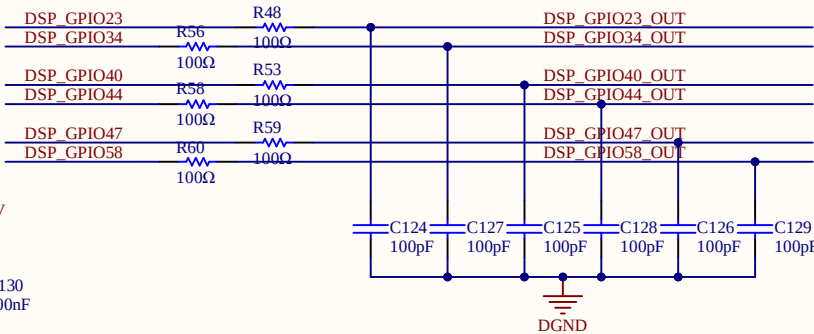
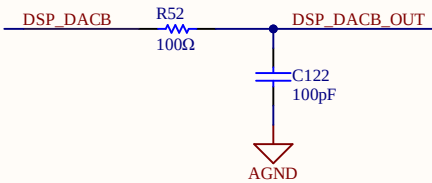
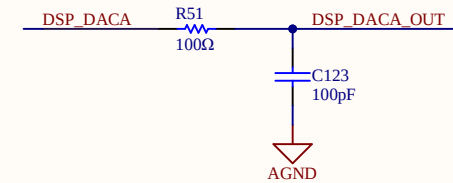
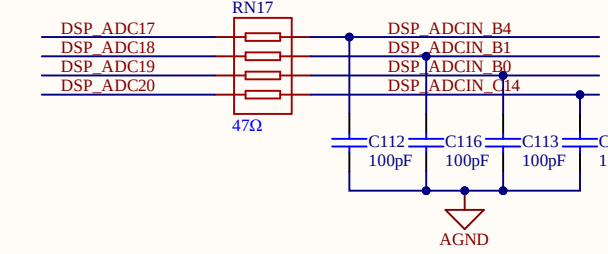
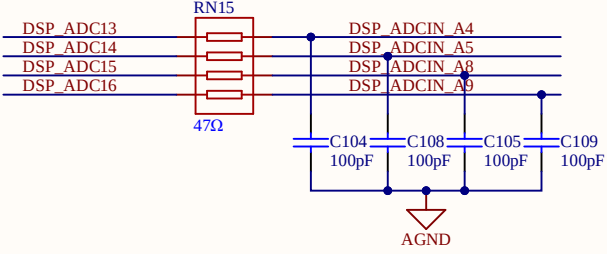
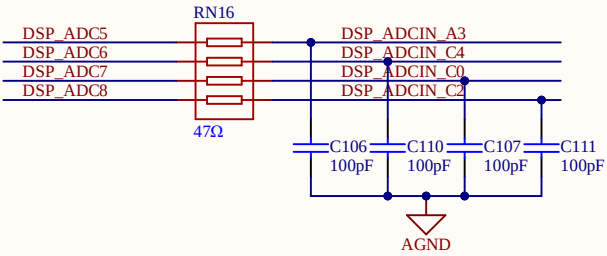
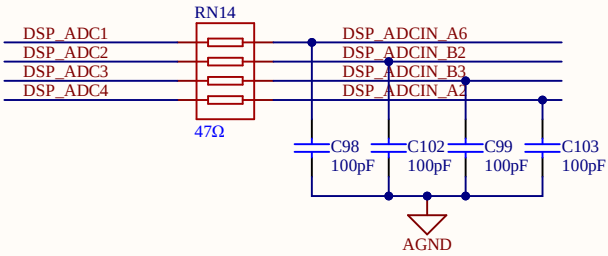
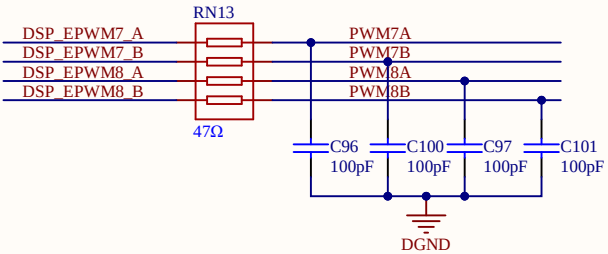
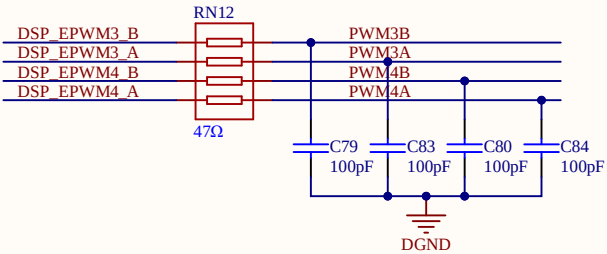
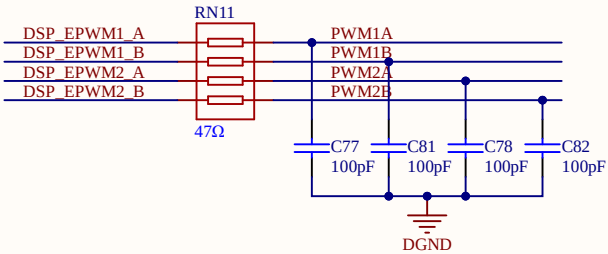


Title		
Size	Number	Revision
A4		
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\...\DSP_Core.SchDoc	Drawn By:

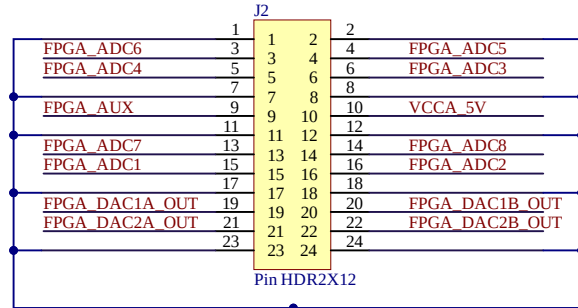


Title		
Size A4	Number	Revision
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\...\Peripheral_utilities_SchDown By:	

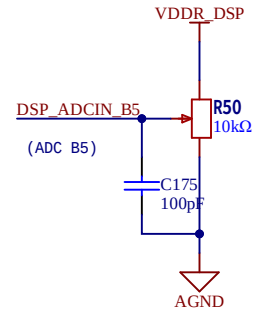
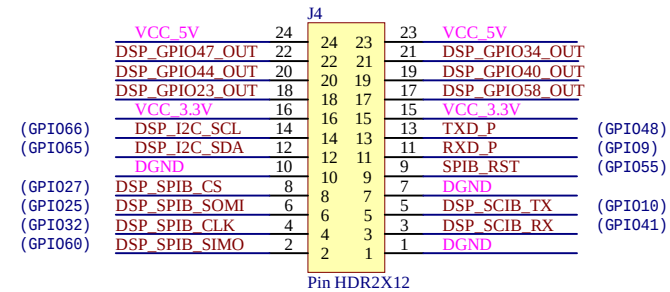
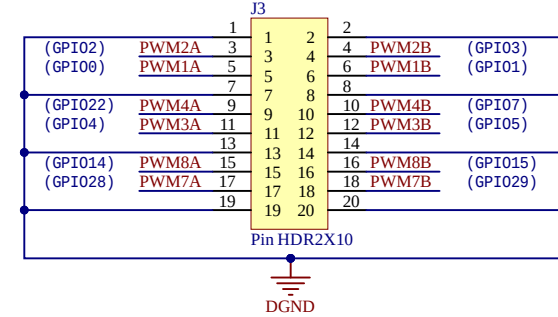
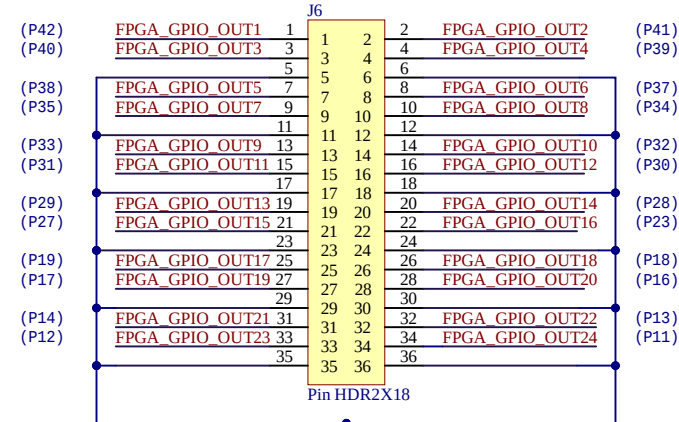
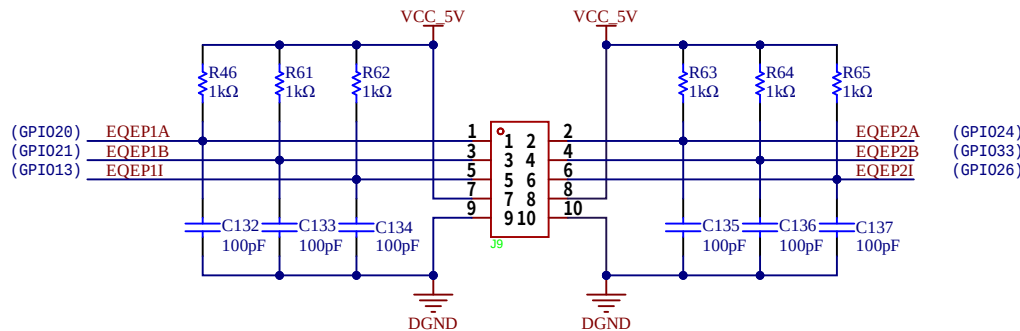
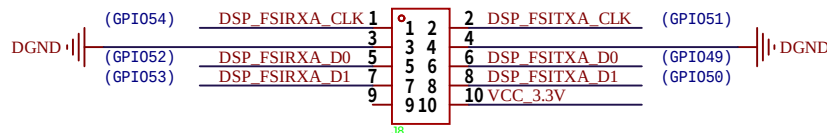
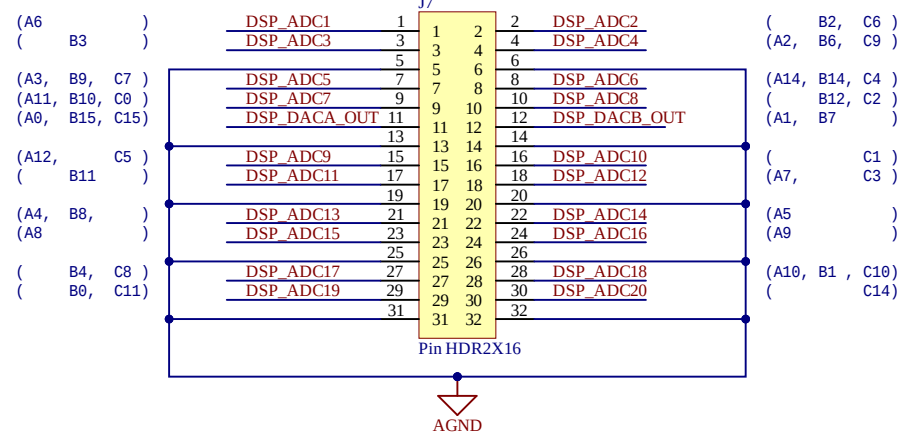
通过RC进行信号过滤，提高质量(远距离)



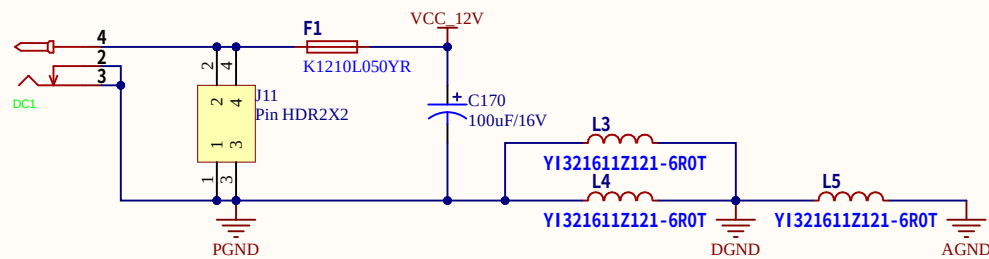
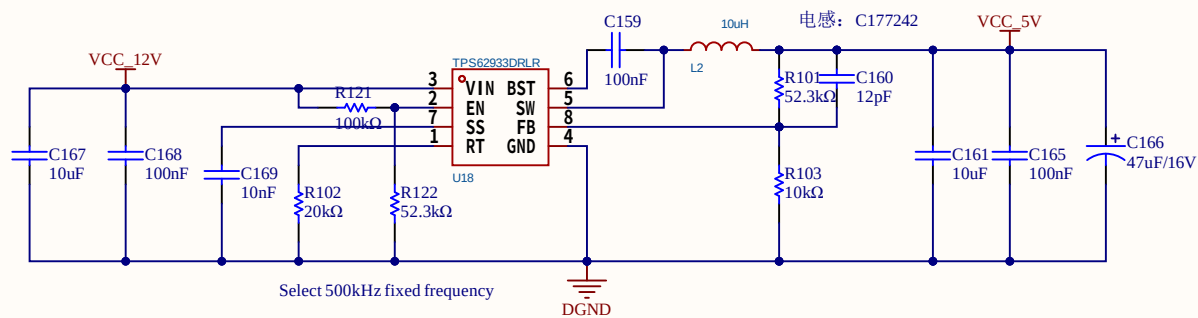
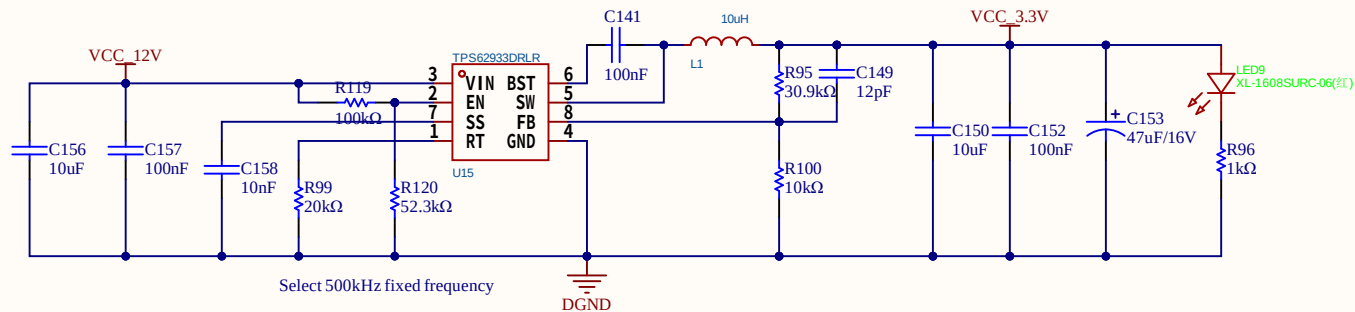
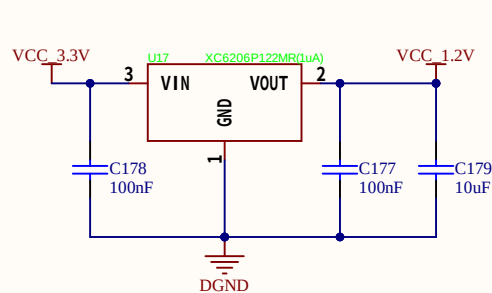
Title		
Size	Number	Revision
A4		
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\...\DSP_Filter_Matrix.Sch	Drawn By:



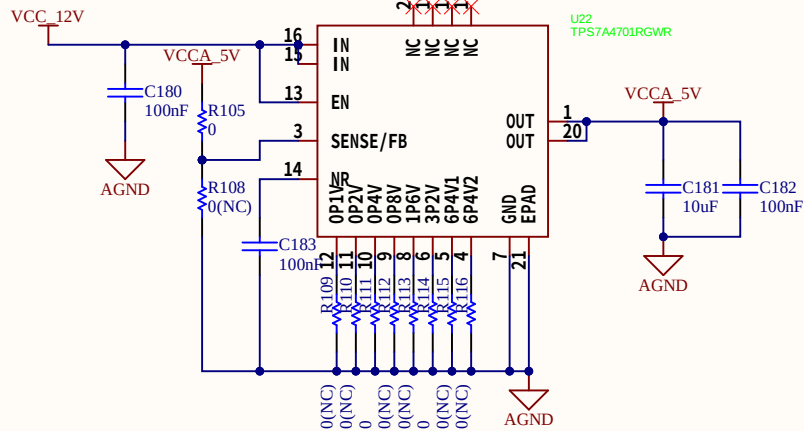
AGND



Title		
Size	Number	Revision
A4		
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\...\nterface_sheet.SchDoc	Drawn By:



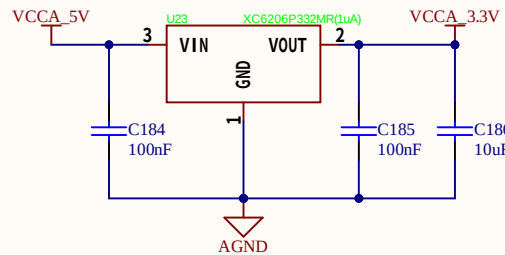
Title		
Size	Number	Revision
A4		
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\...\Aux_power.SchDoc	Drawn By:



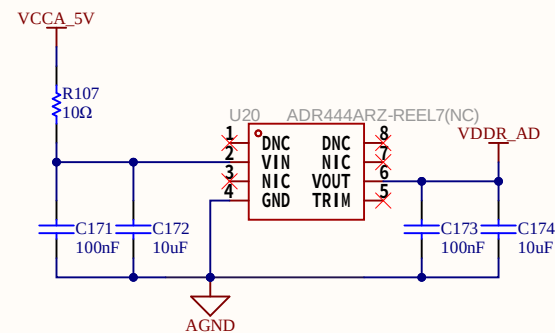
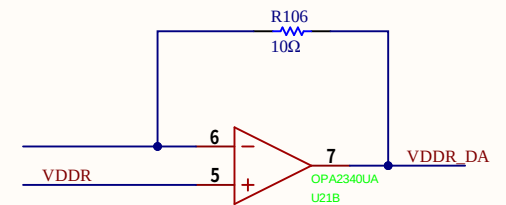
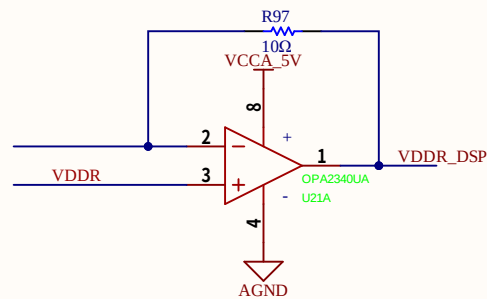
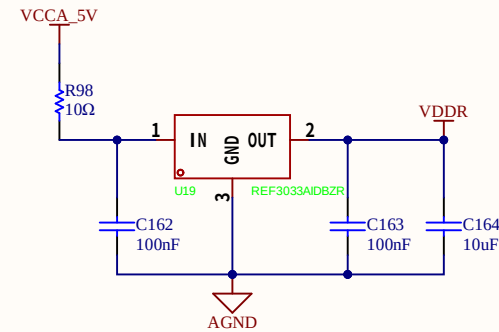
12V到5V模拟电源

Table 2. Common Output Voltages and Corresponding Pin Settings

V _O (V)	PIN NAMES AND VOLTAGE PER PIN							
	0P1V 100 mV	0P2V 200 mV	0P4V 400 mV	0P8V 800 mV	1P6V 1.6 V	3P2V 3.2 V	6P4V1 6.4 V	6P4V2 6.4 V
1.4	Open	Open	Open	Open	Open	Open	Open	Open
1.5	GND	Open	Open	Open	Open	Open	Open	Open
1.8	Open	Open	GND	Open	Open	Open	Open	Open
2.5	GND	GND	Open	GND	Open	Open	Open	Open
3	Open	Open	Open	Open	GND	Open	Open	Open
3.3	GND	GND	Open	Open	GND	Open	Open	Open
4.5	GND	GND	GND	GND	GND	Open	Open	Open
5	Open	Open	GND	Open	Open	GND	Open	Open
10	Open	GND	GND	Open	GND	Open	GND	Open
12	Open	GND	GND	Open	GND	GND	GND	Open
15	Open	Open	Open	GND	Open	Open	GND	GND
18	Open	GND	GND	Open	Open	GND	GND	GND
20.5	GND	GND	GND	GND	GND	GND	GND	GND



这个电源给DAC供电，可以放在DAC旁边。



Title		
Size	Number	Revision
A4		
Date:	3/04/2026	Sheet of
File:	F:\OneDriveSync\Aux_Analog_PowerSupply	Drawn By:

A

B

C

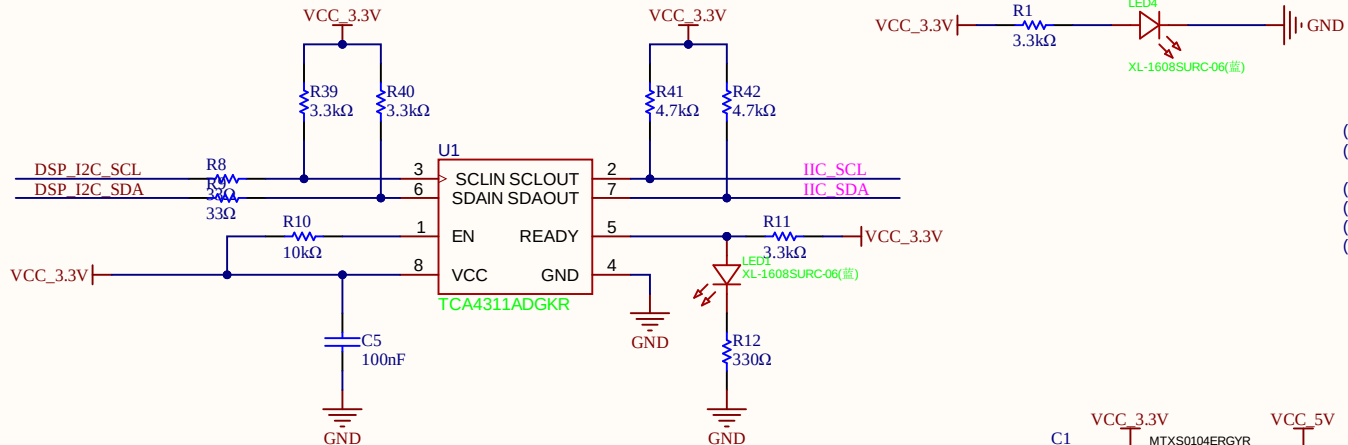
D

A

B

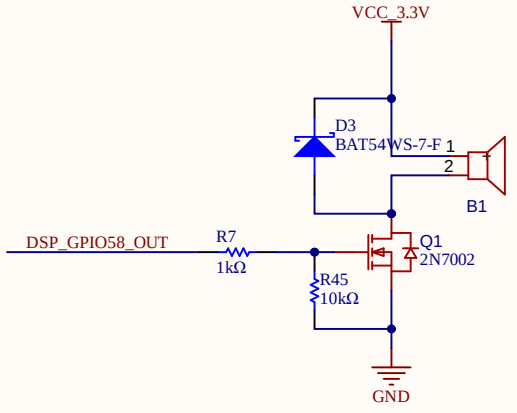
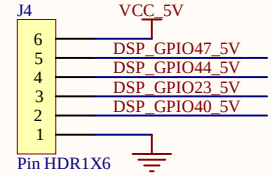
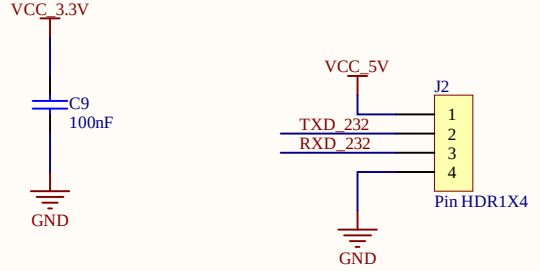
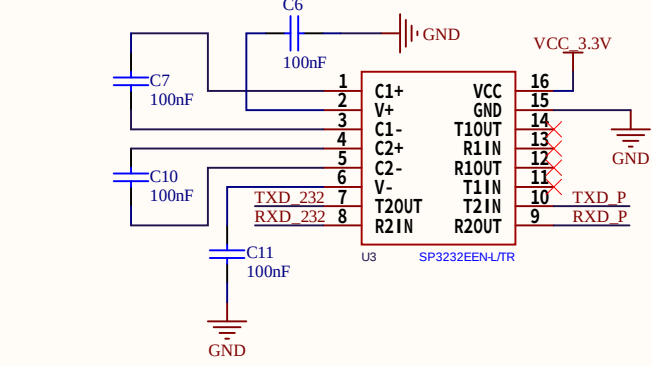
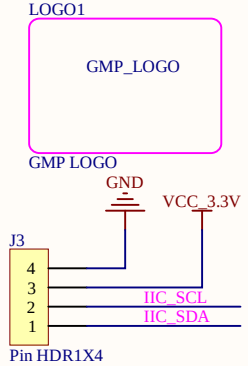
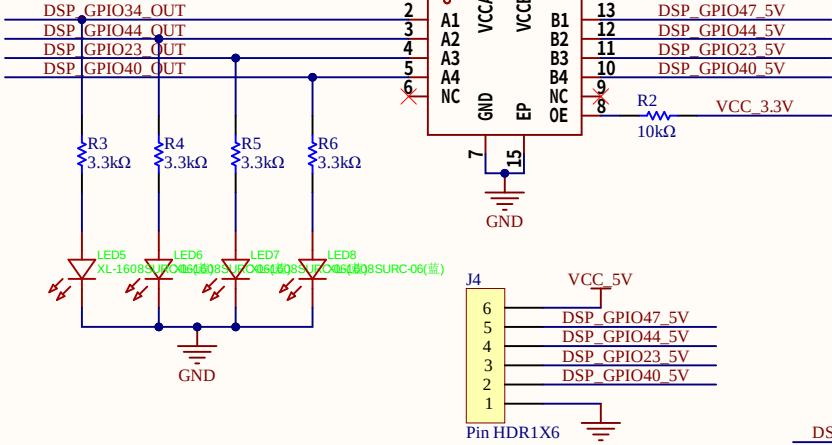
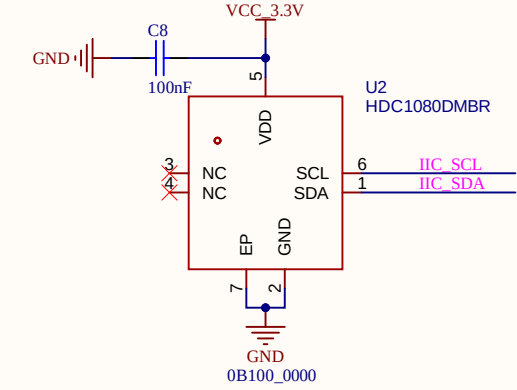
C

D



VCC 5V		24	24	23	23	VCC 5V
DSP_GPIO47_OUT	22	22	21	21	DSP_GPIO34_OUT	
DSP_GPIO44_OUT	20	20	19	19	DSP_GPIO40_OUT	
DSP_GPIO23_OUT	18	18	17	17	DSP_GPIO58_OUT	
VCC 3.3V	16	16	15	15	VCC 3.3V	
(GPIO66)	DSP_I2C_SCL	14	14	13	TXD P	(GPIO48)
(GPIO65)	DSP_I2C_SDA	12	12	11	RXD P	(GPIO9)
GND	GND	10	10	9	SPIB_RST	(GPIO55)
(GPIO27)	DSP_SPIB_CS	8	8	7	GND	
(GPIO25)	DSP_SPIB_SOMI	6	6	5	DSP_SCIB_TX	(GPIO10)
(GPIO32)	DSP_SPIB_CLK	4	4	3	DSP_SCIB_RX	(GPIO41)
(GPIO60)	DSP_SPIB_SIMO	2	2	1	GND	

Pin HDR2X12



Title		
Size	Number	Revision
A4		
Date:	4/22/2026	Sheet of
File:	F:\OneDriveSync\IIC_Interface.SchDoc	Drawn By:

A

A

B

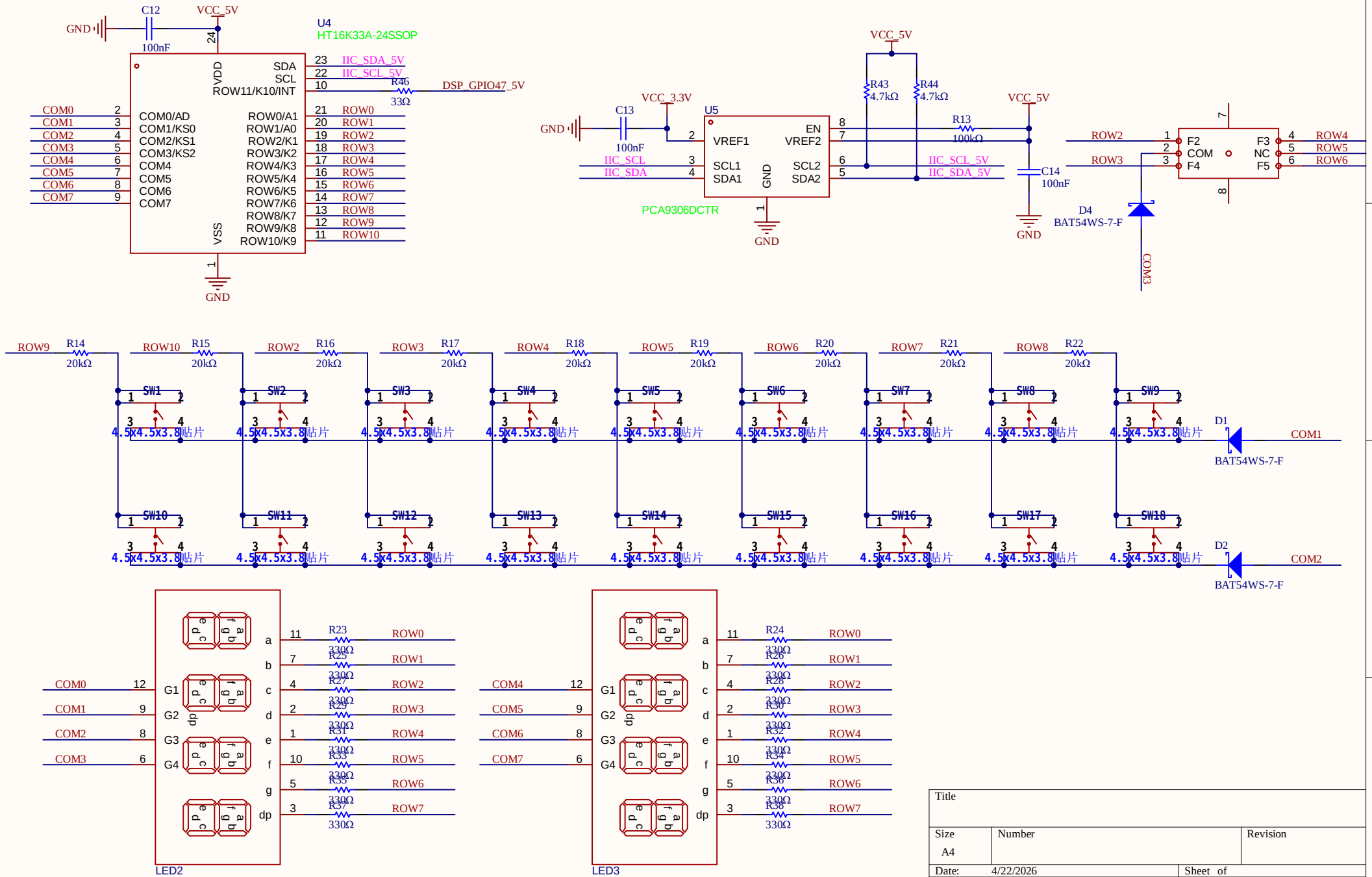
B

C

C

D

D



Title		
Size	Number	Revision
A4		
Date:	4/22/2026	Sheet of
File:	F:\OneDriveSync\LED_7Seg.SchDoc	Drawn By: